HELLO PROJECT · PROJECT GUIDE VOL. 01

KASIRKITA

DARI IDE, JADI APLIKASI.

Buku Panduan Komprehensif Rekayasa Perangkat Lunak: Dari Dokumen Persiapan, Perancangan Proses Bisnis & Data, Pemrograman Desktop Console C17 dengan `lib.h`, Perancangan UI/UX ASCII Art, hingga Kolaborasi Tim Git & GitHub.

NAMA PROYEK
KasirKita (Point of Sale Toko Alat Tulis)

LINGKUNGAN & BAHASA
Bahasa C (C17) · Windows Console API · IntelliJ IDEA

PENYIMPANAN DATA
Pemrosesan Berkas Teks Terstruktur (File Processing .txt)

EDISI & STATUS
Edisi Lengkap Terverifikasi · 2026

HELLO PROJECT / Belajar dengan Membangun.

PROJECT GUIDE · VOL. 01 © 2026

© 2026 Hello Project · Hak Cipta Dilindungi

PENGANTAR

Kata Pengantar & Ringkasan Eksekutif

Buku ini merupakan kompilasi dan dokumentasi komprehensif dari seluruh materi kurikulum **Hello Project: KasirKita**. Dirancang secara khusus untuk membimbing mahasiswa dan pengembang pemula dalam menempuh perjalanan rekayasa perangkat lunak secara utuh—dimulai dari penemuan masalah nyata di dunia usaha, perumusan dokumen kebutuhan, pemodelan proses bisnis dan basis data, implementasi kode tingkat rendah bahasa C pada console terminal Windows menggunakan pustaka `lib.h`, perancangan antarmuka visual ASCII art, hingga standarisasi kerja kolaboratif kelompok menggunakan Git dan GitHub.

Filosofi Pembelajaran: "Belajar dengan Membangun" (Build to Learn)
Banyak pembelajar pemrograman terjebak dalam *tutorial hell*—menghafal sintaks bahasa tanpa memahami bagaimana merancang sistem dari nol. Di KasirKita, Anda tidak sekadar mengetik kode C, melainkan memposisikan diri sebagai insinyur perangkat lunak profesional yang bertanggung jawab atas seluruh siklus hidup sistem: dari kebutuhan pemilik toko hingga pencetakan nota fisik.

Struktur Kurikulum 4 Modul Inti

01. Dokumen Persiapan

Penentuan tema toko alat tulis, pemisahan master vs transaksi, batasan scope standalone, analisis platform desktop console, perumusan Activity Plan (AP), serta pemodelan Probis, Flowchart, ERD, PDM 3NF, dan DFD.

02. Dokumen After

Penyusunan dokumen paska-implementasi: Matriks User Acceptance Testing (UAT), penelusuran cacat/bug, Proposal Proyek, Product Documentation teknis, serta User Manual operasional kasir.

03. Penggunaan lib.h

Penguasaan pustaka manipulasi console Windows: koordinat (x,y), teks, bingkai (border), kursor, pembersihan area, pembacaan file, fungsi input khusus terproteksi, serta audit koreksi bug kritis.

04. UI/UX dengan lib.h

Transformasi citra digital menjadi ASCII Art via ASCIIArt.eu, penataan tata letak visual terminal 80×25, pembuatan header toko, formulir kasir, keranjang dinamis, serta nota belanja profesional.

Modul Ekstra: Panduan Kolaborasi Tim Git & GitHub
Dilengkapi panduan praktis pengelolaan repositori bersama, strategi *Feature Branching*, siklus harian push/pull, anatomi resolusi *Merge Conflict* dengan kepala dingin, serta template `.gitignore` standar IntelliJ IDEA untuk proyek C.

© 2026 Hello Project · Hak Cipta Dilindungi

NAVIGASI DOKUMEN

Daftar Isi Lengkap

BAB 1: PENDAHULUAN & SIKLUS HIDUP PROYEK (SDLC) 6

1.1 Mengenal Hello Project & Studi Kasus KasirKita 6

1.2 Siklus Hidup Perangkat Lunak (SDLC) KasirKita 6

1.3 Peta Jalan Pembelajaran (Roadmap 4 Modul) 7

BAB 2: MODUL 01 — DOKUMEN PERSIAPAN & REKAYASA KEBUTUHAN 8

2.1 Menemukan Masalah Nyata & Menentukan Tema Proyek 8

2.2 Analisis Platform: Desktop Console C vs Web & Basis Data Server 8

2.3 Arsitektur Standalone Offline vs Sistem Terintegrasi 9

2.4 Menetapkan Batasan Cakupan (Scope: In-Scope vs Out-of-Scope) 9

2.5 Prinsip Pemisahan Data: Entitas Master vs Transaksi 10

2.6 Ekstraksi Kebutuhan & Identifikasi Atribut Data 11

2.7 Activity Plan (AP): Format Rencana Kerja & Penjadwalan Tim 11

BAB 3: ANALISIS PROSES BISNIS, PEMODELAN DATA & DFD (PROBIS) 13

3.1 Analisis Proses Bisnis: Kondisi As-Is vs Solusi To-Be 13

3.2 Diagram Alir Transaksi Penjualan (Flowchart Checkout Kasir) 13

3.3 Pemodelan Konseptual: ERD Awal Berbasis Nota Fisik 14

3.4 Normalisasi Basis Data Fisik (PDM: 1NF, 2NF, hingga 3NF) 15

3.5 Denormalisasi Terkontrol: Snapshot Harga & Penyimpanan Total 16

3.6 Data Flow Diagram (DFD): Level 0 (Konteks) & Level 1 (Dekomposisi) 17

3.7 Kamus Data Komprehensif (Data Dictionary) 18

3.8 Desain Penyimpanan Berkas Teks (.txt) pada Bahasa C 21

3.9 Matriks Fitur & Kriteria Keberhasilan (Acceptance Criteria) 21

BAB 4: MODUL 02 — DOKUMEN AFTER (PENGUJIAN & SERAH TERIMA) 23

4.1 Metodologi User Acceptance Testing (UAT) Kasir 23

4.2 Matriks Skenario UAT: Kasus Positif, Negatif, & Batas 23

4.3 Pelacakan Cacat (Bug Hunter & Defect Management) 25

© 2026 Hello Project · Hak Cipta Dilindungi

4.4 Panduan Penyusunan Proposal Proyek Perangkat Lunak 26
4.5 Product Documentation (Spesifikasi Teknis & Database) 27
4.6 User Guide / User Manual KasirKita 27
4.7 Format Laporan Akhir & Checklist Serah Terima 28

BAB 5: MODUL 03 — PEMROGRAMAN CONSOLE WINDOWS DENGAN LIB.H 29

5.1 Arsitektur Antarmuka Console Windows API & Peran lib.h 29
5.2 Standar Struktur Folder Proyek C di IntelliJ IDEA 29
5.3 Persiapan Compiler MinGW-w64 & Konfigurasi Standar C17 30
5.4 Sistem Koordinat Console Windows (Sumbu X & Y, Grid 80×25) 31
5.5 Pustaka Fungsi Posisi & Teks (gotoxy, gotoPrintString, gotoPrintChar) 31
5.6 Pustaka Fungsi Bingkai & Area (borderLine, borderDoubleLine, clearArea) 31
5.7 Manajemen Visibilitas Kursor & Jendela (hideCursor, fullscreen, setConsoleFont) 32
5.8 Pembacaan File Teks Eksternal (PrintTxt) 32
5.9 Pustaka Fungsi Input Interaktif (String, Number, Money, Password) 32
5.10 Analisis Masalah Kritis & Solusi Bug Fixes pada lib.h Asli 33

BAB 6: REFERENSI LENGKAP API LIB.H (18 FUNGSI) 35

6.1 Dokumentasi Detail Seluruh Fungsi Console Windows 35
6.2 Tabel Konstanta Keyboard & Tombol Panah Navigasi 39
6.3 Contoh Program Menu Console Lengkap Siap Eksekusi 40

BAB 7: MODUL 04 — DESAIN UI/UX TERMINAL CONSOLE & ASCII ART 43

7.1 Prinsip Desain UI/UX Antarmuka Teks (TUI) 43
7.2 Persiapan & Sketsa Gambar untuk ASCII Art 43
7.3 Panduan Konversi Citra ke Karakter via ASCIIArt.eu 43
7.4 Teknik Integrasi ASCII Art pada Program C (Escaping Rules) 44
7.5 Desain Tata Letak Layar Kasir: Banner, Menu, Form, & Nota Belanja 45
7.6 Simulator Terminal Studio 80×25 & Uji Keterbacaan Antarmuka 46

BAB 8: MODUL KOLABORASI — PANDUAN KERJA TIM DENGAN GIT & GITHUB 47

8.1 Urgensi Sistem Kontrol Versi (VCS) dalam Kerja Kelompok 47
8.2 Memahami 4 Wilayah Kerja Git (Working, Staging, Local, Remote) 47

© 2026 Hello Project · Hak Cipta Dilindungi

8.3 Setup Identitas & Inisialisasi Repository Bersama 48
8.4 Strategi Feature Branching: Bekerja Paralel Tanpa Tabrakan 48
8.5 Rutinitas Harian: Aturan Emas Push, Pull, dan Commit 48
8.6 Anatomi & Resolusi Merge Conflict dengan Kepala Dingin 49
8.7 Standar Pull Request (PR) & Praktik Terbaik Code Review 50
8.8 Template .gitignore Standar C di IntelliJ IDEA 50
8.9 Git Emergency Cheat Sheet & Penanganan Kondisi Darurat 51

BAB 9: LAMPIRAN & KODE SUMBER REFERENSI 53

9.1 Kode Lengkap Header lib.h Terkoreksi (Clean Patched Version) 53
9.2 Skrip Build Otomatis PowerShell & Batch 57
9.3 Glosarium Istilah Rekayasa Perangkat Lunak 57

© 2026 Hello Project · Hak Cipta Dilindungi

PENDAHULUAN

BAB 1: Pendahuluan & Siklus Hidup Proyek (SDLC)

1.1 Mengenal Hello Project & Studi Kasus KasirKita

KasirKita adalah proyek percontohan aplikasi kasir desktop (*Point of Sale - POS*) untuk toko alat tulis kantor (ATK). Aplikasi ini dibangun menggunakan bahasa **C (standar C17)** di lingkungan IDE **IntelliJ IDEA / CLion / MinGW**, memanfaatkan pustaka antarmuka console Windows `lib.h` dan menggunakan sistem penyimpanan data berbasis pemrosesan berkas teks terstruktur (*file processing* `.txt`).

Dalam proses operasionalnya, pemilik toko mengelola katalog produk, kategori, dan stok barang, sedangkan kasir bertugas mencatat transaksi pembelian tunai dari pelanggan, memeriksa ketersediaan stok secara otomatis, menghitung total belanja serta uang kembalian, mencetak struk nota belanja, dan memperbarui catatan persediaan barang secara persisten ke dalam berkas.

Masalah yang Diselesaikan

Pencatatan manual berbasis nota kertas di toko ATK sering menimbulkan ketidaksesuaian stok, risiko kesalahan perhitungan kembalian, hilangnya bukti transaksi, serta memakan waktu berjam-jam bagi pemilik toko untuk merekap omzet harian.

Hasil Akhir yang Dibangun

Program desktop console mandiri (*standalone executable*) yang mampu memproses transaksi kasir, memvalidasi integritas data secara otomatis, menyajikan antarmuka terminal bergaya retro dengan ASCII art, dan mencatat transaksi ke dalam berkas lokal.

1.2 Siklus Hidup Perangkat Lunak (SDLC) KasirKita

Banyak pemula beranggapan bahwa pembuatan perangkat lunak dimulai langsung dengan mengetik baris kode (*coding*). Pandangan ini keliru dan sering kali berujung pada kegagalan proyek kelompok. Perangkat lunak yang handal lahir dari tahapan yang terukur melalui **Software Development Life Cycle (SDLC)**.

© 2026 Hello Project · Hak Cipta Dilindungi

Fase SDLC	Aktivitas Utama	Dokumen & Artefak Luaran	Peran dalam Tim
1. Requirement & Planning	Mengidentifikasi masalah toko, menentukan batas lingkup (scope), memisahkan data master vs transaksi, menyusun jadwal.	Activity Plan (AP), Dokumen Persiapan, Daftar Kebutuhan Fungsional.	Project Manager, Business Analyst
2. System Analysis & Design	Memetakan alur bisnis (As-is vs To-be), merancang Flowchart logika checkout atomik, memodelkan ERD, normalisasi PDM 3NF, menyusun DFD.	Dokumen Proses Bisnis (Probis), Diagram Flowchart, Skema ERD & PDM, Kamus Data.	System Architect, Database Designer
3. UI/UX Design	Merancang tata letak layar terminal (80×25 kolom), membuat sketsa banner toko, mengonversi gambar ke ASCII art.	Aset Teks ASCII Art, Sketsa Grid Layar Menu, Desain Format Struk Nota.	UI/UX Designer
4. Implementation (Coding)	Menata struktur folder, mengatur header `lib.h`, mengimplementasikan logika navigasi, validasi input, dan file I/O C.	Source code `main.c`, header `lib.h` yang ditambal, berkas data `.txt`.	Programmer / Software Engineer
5. Testing & Acceptance	Menjalankan skenario pengujian bersama calon pengguna nyata (UAT), mencatat bug pada kasus negatif dan batas uang.	Dokumen User Testing (UT), Catatan Perbaikan Cacat (Bug Log), Rekapitulasi Audit.	QA Tester, Kasir/Pemilik Toko
6. Documentation & Handover	Menyusun dokumentasi teknis sistem, buku petunjuk penggunaan kasir, dan laporan akhir proyek.	Product Documentation, User Manual, Proposal Proyek, Laporan Akhir.	Technical Writer, Seluruh Anggota

> **Penting: Jalur Perbaikan (Feedback Loop)**
> SDLC bukanlah proses searah yang kaku. Ketika pengujian di Tahap 5 menemukan kegagalan (misalnya uang kurang tapi nota tetap tercetak), tim wajib kembali ke Tahap 2 (memeriksa flowchart keputusan) dan Tahap 4 (memperbaiki logika transaksi di kode C). Dokumentasi harus selalu diperbarui agar mencerminkan implementasi aktual!

1.3 Peta Jalan Pembelajaran (Roadmap 4 Modul)

Kurikulum Hello Project dirancang secara berjenjang menjadi 4 modul praktis yang saling menopang:

Modul 01: Dokumen Persiapan

Fondasi rekayasa perangkat lunak. Memastikan tim memiliki kesepahaman bulat mengenai apa yang dibangun sebelum menyentuh compiler C.

Modul 02: Dokumen After

Pertanggungjawaban produk. Menyediakan bukti otentik pengujian kasir, manual operasional toko, dan spesifikasi arsitektur produk.

Modul 03: Penggunaan lib.h

Eksplorasi teknis tingkat rendah. Mengendalikan kursor layar, jendela console Windows, kotak bingkai, dan input data terproteksi.

Modul 04: UI/UX dengan lib.h

Estetika visual antarmuka teks. Memadukan keindahan seni karakter ASCII dengan kekuatan komputasi bahasa pemrograman C.

© 2026 Hello Project · Hak Cipta Dilindungi

MODUL 01: DOKUMEN PERSIAPAN

BAB 2: Dokumen Persiapan & Rekayasa Kebutuhan

2.1 Menemukan Masalah Nyata & Menentukan Tema Proyek

Langkah paling fundamental dalam rekayasa perangkat lunak adalah mengidentifikasi masalah nyata yang dihadapi oleh pengguna akhir. Aplikasi yang dibangun tanpa pemahaman masalah yang tepat sering kali menghasilkan perangkat lunak yang canggih secara teknis namun tidak berguna di lapangan.

Pada studi kasus **KasirKita**, kita membedah operasional sebuah toko alat tulis kantor (ATK) skala kecil hingga menengah.

Komponen Analisis	Deskripsi pada KasirKita
Masalah Utama	Pencatatan penjualan nota kertas manual menyebabkan selisih persediaan fisik dengan catatan stok, rawan kekeliruan perhitungan uang kembalian pada jam sibuk, serta memakan waktu berjam-jam bagi pemilik toko untuk merekap omzet harian.
Pengguna Sistem (Aktor)	1. **Kasir:** Melayani transaksi belanja langsung pembeli, memasukkan barang ke keranjang, memvalidasi uang tunai, dan menyerahkan nota. 2. **Pemilik Toko:** Memperbarui daftar katalog barang dan harga, memantau persediaan stok sisa, dan memeriksa rekap laporan keuangan harian.
Tujuan Proyek	Membangun sistem kasir terkomputerisasi yang mampu mencatat transaksi secara atomik, memperbarui stok secara otomatis, mencetak bukti transaksi yang transparan, dan dapat diverifikasi kembali sewaktu-waktu.
Skenario Acuan Standar	*"Pelanggan Rani membeli 2 Buku Tulis A5 dengan harga satuan Rp12.000. Total belanja adalah Rp24.000. Rani membayar dengan uang tunai Rp30.000. Sistem menghitung uang kembalian Rp6.000. Stok Buku Tulis A5 di berkas katalog secara otomatis berkurang dari 20 menjadi 18 unit."* Angka-angka pada skenario konkret ini menjadi tolok ukur pengujian kebenaran logika dari fase analisis hingga fase pengkodean.

2.2 Analisis Platform: Desktop Console C vs Web & Basis Data Server

Sebelum menulis kode, tim pengembang harus mengambil keputusan arsitektural mengenai platform yang tepat. Mengapa KasirKita menggunakan aplikasi desktop berbasis console terminal bahasa C dengan pemrosesan berkas teks (*file processing*), dan bukan aplikasi web modern dengan database server?

© 2026 Hello Project · Hak Cipta Dilindungi

Aspek Evaluasi	Desktop Console + File Processing (C17)	Web Application + Database Server (SQL)
Cara Akses	Dijalankan langsung sebagai program executable lokal (`.exe`) di meja kasir melalui jendela terminal Windows.	Dibuka menggunakan web browser (Chrome, Edge); memerlukan web server (Node.js/Apache/Nginx).
Mekanisme Penyimpanan	Berkas teks lokal terstruktur (`katalog.txt`, `penjualan.txt`). Mudah dibaca dan tidak memerlukan server DBMS terpisah.	Database server relasional (MySQL, PostgreSQL, Oracle). Memerlukan konfigurasi port, koneksi jaringan, dan service database aktif.
Konektivitas Jaringan	**100% Offline:** Beroperasi sempurna tanpa koneksi internet maupun jaringan lokal. Sangat cocok untuk toko kelontong atau kios.	Sangat bergantung pada stabilitas jaringan internet atau LAN untuk komunikasi client-server. Rawan offline jika sinyal putus.
Kecepatan & Konsumsi Memori	Sangat ringan (konsumsi RAM < 15 MB), instan saat dibuka, tanpa overhead browser engine maupun runtime besar.	Memerlukan alokasi RAM besar untuk browser dan DBMS server (seringkali > 500 MB).
Kompleksitas Lingkungan	Cukup menyalin file binary executable dan folder data ke laptop kasir; portabel tanpa instalasi rumit.	Memerlukan instalasi runtime, migrasi database, manajemen sertifikat SSL, dan pemeliharaan dependensi berkala.
Pencetakan Bukti Transaksi	Mencetak struk teks langsung ke printer thermal POS melalui port serial/USB atau menampilkan preview teks terformat di layar.	Melalui dialog cetak browser (`window.print()`) yang sering memakan waktu render CSS lebih lambat.

2.3 Arsitektur Standalone Offline vs Sistem Terintegrasi

Arsitektur sistem KasirKita adalah **Standalone System**. Artinya, program beroperasi secara mandiri di satu komputer kasir tanpa mengirim atau meminta data dari server luar. Keputusan ini diambil untuk menjamin keandalan tertinggi: mesin kasir tidak boleh macet hanya karena jaringan internet sedang lambat atau router Wi-Fi toko mengalami *restart*.

Karakteristik Sistem Standalone:

- **Otonom:** Seluruh aturan bisnis, validasi, dan penyimpanan berada pada satu bundel executable lokal.
- **Integritas Data Lokal:** Kegagalan komputer kasir hanya berdampak pada kasir tersebut, tidak merusak jaringan toko lainnya.
- **Pencadangan Sederhana:** Backup data cukup dilakukan dengan menyalin berkas teks `.txt` atau `.dat` ke media penyimpanan eksternal (flashdisk).

2.4 Menetapkan Batasan Cakupan (Scope)

Salah satu jebakan terbesar yang menggagalkan proyek perangkat lunak adalah *Scope Creep*—keinginan untuk terus menambah fitur baru di tengah masa pengembangan tanpa memperhitungkan batas waktu dan kapasitas tim. Oleh karena itu, batasan cakupan wajib dikunci sejak Modul 01.

© 2026 Hello Project · Hak Cipta Dilindungi

Termasuk dalam Lingkup (In-Scope)

✓ Pengelolaan katalog produk, kategori barang, dan stok fisik.
✓ Pencatatan identitas pelanggan secara opsional (bisa kasir umum).
✓ Keranjang belanja interaktif (tambah barang, hitung subtotal otomatis).
✓ Proses checkout tunai dengan validasi ketersediaan stok & kecukupan uang.
✓ Transaksi atomik (pembaharuan stok bersamaan dengan pencetakan nota).
✓ Format nota belanja ASCII yang rapi untuk struk belanja.
✓ Rekapitulasi laporan keuangan dan omzet harian toko.
✓ Penyimpanan data persisten berbasis file teks terstruktur.
✓ Pengujian penerimaan pengguna (UAT) dan dokumentasi lengkap.

Di Luar Lingkup (Out-of-Scope)

✗ Multi-cabang toko dan sinkronisasi basis data cloud.
✗ Pembayaran digital (QRIS, kartu debit/kredit, e-wallet, payment gateway).
✗ Perhitungan pajak pertambahan nilai (PPN) dan kupon diskon dinamis.
✗ Retur barang rusak atau pembatalan nota yang sudah dicetak.
✗ Manajemen pembelian barang ke pemasok (supplier purchase order).
✗ Sistem autentikasi pengguna berlapis (Role-Based Access Control produksi).
✗ Migrasi ke antarmuka grafis (GUI) modern seperti Qt atau WinForms.

2.5 Prinsip Pemisahan Data: Entitas Master vs Transaksi

Dalam merancang sistem informasi penjualan, seorang insinyur wajib memahami secara mendalam perbedaan antara **Data Master** dan **Data Transaksi**. Mencampuradukkan kedua jenis data ini adalah sumber utama redundansi dan anomali pada sistem.

Dimensi Pembeda	Entitas Master (Master Data)	Entitas Transaksi (Transaction Data)
Definisi Konseptual	Data acuan dasar yang menjadi patokan bisnis dan digunakan berulang kali dalam berbagai proses operasional.	Catatan historis dari peristiwa atau kejadian bisnis tertentu yang terjadi pada suatu titik waktu.
Contoh di KasirKita	• `KATEGORI` (Alat Tulis, Kertas, Buku) • `PRODUK` (Buku Tulis A5, Pulpen Gel 0.5) • `PELANGGAN` (Rani, Budi, Pelanggan Umum)	• `PENJUALAN` (Nota #001 pada tanggal 2026-10-20) • `DETAIL PENJUALAN` (2 pcs Buku Tulis A5 @ Rp12.000)
Frekuensi Perubahan	**Jarang berubah (statis).** Hanya diperbarui ketika pemilik toko menambah item baru atau mengubah harga pokok.	**Sangat dinamis dan bertambah terus.** Setiap kali ada pelanggan yang membayar, baris transaksi baru tercipta.
Karakteristik Integritas	Dapat diubah atau dinonaktifkan dengan hati-hati. Menghapus data master yang sudah pernah ditransaksikan dilarang keras.	**Bersifat Immutable (Kekal).** Sekali transaksi disahkan (*commit*), data tidak boleh diedit karena mencerminkan sejarah audit uang masuk.
File Penyimpanan	Disimpan pada `data/katalog.txt` dan `data/kategori.txt`.	Disimpan pada `data/penjualan.txt` dan `data/penjualan_detail.txt`.

© 2026 Hello Project · Hak Cipta Dilindungi

2.6 Ekstraksi Kebutuhan & Identifikasi Atribut Data

Dari skenario bisnis transaksi kasir, kita dapat mengekstrak atribut-atribut data esensial yang diperlukan untuk membangun basis data fisik:

- **Kategori:** Memerlukan `id_category` (kode unik penanda) dan `name` (nama kategori barang).
- **Produk:** Memerlukan `id_product`, `id_category` (relasi ke kategori), `sku` (kode unik barcode barang, contoh: P001), `name` (nama barang), `price` (harga jual satuan dalam rupiah bulat), dan `stock` (jumlah fisik yang tersedia).
- **Pelanggan:** Memerlukan `id_customer`, `name` (nama pembeli), dan `phone` (nomor kontak opsional).
- **Penjualan (Header Nota):** Memerlukan `id_sale` (nomor nota unik), `id_customer` (relasi ke pembeli), `created_at` (stempel waktu transaksi ISO-8601), `total` (total tagihan belanja), dan `paid` (nominal uang tunai yang diserahkan).
- **Item Penjualan (Detail Nota):** Memerlukan `id_sale`, `id_product`, `quantity` (jumlah barang yang dibeli), dan `unit_price` (snapshot harga satuan barang saat detik transaksi berlangsung).

2.7 Activity Plan (AP): Format Rencana Kerja & Penjadwalan Tim

Activity Plan (AP) adalah dokumen manajemen proyek yang merinci seluruh rangkaian kegiatan teknis, pembagian tugas anggota kelompok (PIC - *Person in Charge*), target waktu penyelesaian, luaran yang diserahkan, serta status kemajuan terkini.

> **Klarifikasi Istilah Akademik:**
> Dalam konteks dokumentasi rekayasa perangkat lunak, singkatan **AP** merujuk pada **Activity Plan** (Rencana Kerja Kegiatan), bukan Arsitektur Program! Dokumen ini menjadi kontrak kinerja internal tim kelompok.

No	Nama Aktivitas / Kegiatan	Penanggung Jawab (PIC)	Target Jadwal	Luaran (Deliverable) yang Diserahkan	Status
1	Inisiasi Proyek & Analisis Kebutuhan	Analis Bisnis	Minggu 1	Dokumen Tema, Batasan Scope, Contoh Skenario Transaksi.	Selesai
2	Perancangan Proses Bisnis & Data	System Architect	Minggu 2	Flowchart Checkout, ERD Awal, PDM 3NF, DFD Level 0/1, Kamus Data.	Selesai
3	Eksplorasi & Pengujian Library `lib.h`	Lead Programmer	Minggu 3	Catatan Audit Fungsi, File Latihan Console C, Patch Koreksi Bug Input.	Selesai
4	Perancangan Antarmuka & ASCII Art	UI/UX Designer	Minggu 4	Aset Teks ASCIIArt.eu, Tata Letak Menu Kasir, Template Nota Belanja.	Selesai
5	Integrasi Kode & Pengujian UAT	QA & Seluruh Tim	Minggu 5	Program `kasirkita.exe`, Berkas UAT, Log Pelacakan Bug.	Dalam Proses
6	Penyusunan Dokumentasi & Serah Terima	Technical Writer	Minggu 6	Product Documentation, User Manual, Laporan Akhir Proyek.	Dalam Proses

© 2026 Hello Project · Hak Cipta Dilindungi

Checklist Kesiapan Modul 01:

- ✓ Saya dan tim mampu menjelaskan masalah toko alat tulis dan batasan sistem KasirKita secara gamblang.
- ✓ Saya memahami perbedaan esensial antara entitas master (katalog) dan entitas transaksi (penjualan).
- ✓ Skenario transaksi acuan (Rani: 2 buku @12.000, bayar 30.000, kembali 6.000) telah disepakati seluruh tim.
- ✓ Activity Plan telah dilengkapi dengan nama anggota tim, tenggat waktu, dan luaran nyata.

© 2026 Hello Project · Hak Cipta Dilindungi

MODUL 01 DETAIL: PROBIS & DATA

BAB 3: Analisis Proses Bisnis, Pemodelan Data & DFD

3.1 Analisis Proses Bisnis: Kondisi As-Is vs Solusi To-Be

Pemodelan proses bisnis bertujuan memetakan bagaimana kegiatan operasional berjalan saat ini (*As-Is*) dan bagaimana sistem informasi yang kita kembangkan akan mentransformasi alur kerja tersebut menjadi lebih efektif, terukur, dan aman (*To-Be*).

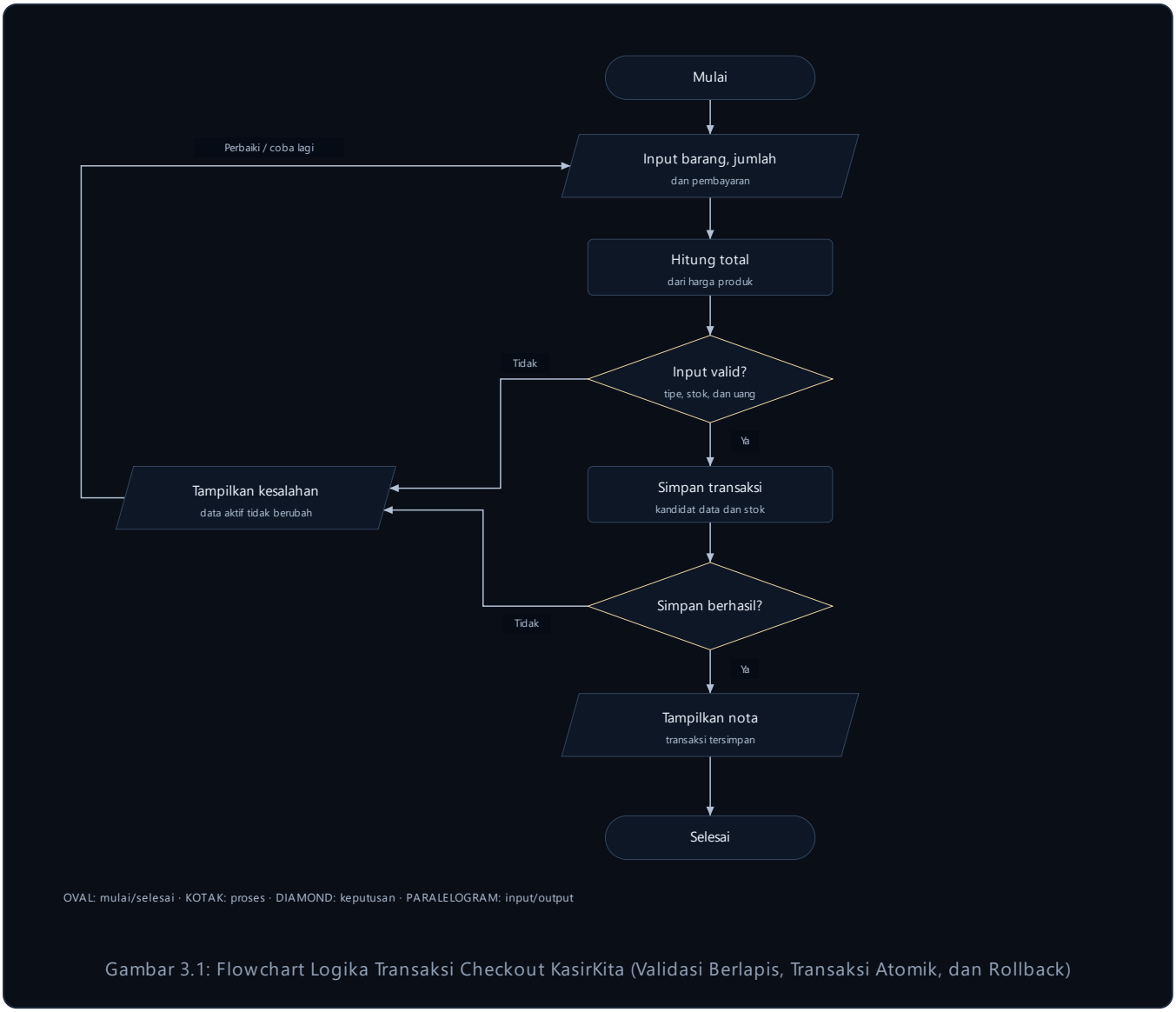
Tahapan Bisnis	Proses Manual Toko Saat Ini (As-Is)	Proses Sistem Digital KasirKita (To-Be)
1. Pemilihan Barang	Kasir mengamati label harga pada barang atau bertanya kepada pemilik toko. Menghitung ketersediaan stok fisik di rak.	Kasir mengetikkan nama barang atau kode SKU pada layar terminal. Sistem langsung menampilkan deskripsi, harga resmi, dan sisa stok terkini.
2. Perhitungan Belanja	Kasir menuliskan item di kertas nota bon rangkap dan menjumlahkan total harga belanja menggunakan kalkulator fisik. Rawan salah tekan angka.	Sistem menghitung subtotal dan total belanja secara otomatis dan presisi hingga bilangan bulat terkecil (Rupiah).
3. Validasi Transaksi	Sering kali uang kembalian baru dihitung setelah nota dirobek. Jika pembeli kekurangan uang, nota kertas harus dicoret-coret atau dibatalkan manual.	Sistem memvalidasi kecukupan pembayaran tunai sebelum transaksi disimpan. Jika uang kurang, sistem menolak eksekusi dan meminta penyesuaian nominal.
4. Pengurangan Persediaan	Pengurangan stok di buku besar toko baru dilakukan malam hari atau akhir bulan. Selisih stok fisik sering baru disadari saat barang sudah habis.	Stok barang di berkas data terstruktur berkurang secara otomatis dan instan begitu pembayaran dinyatakan sah (*atomic update*).
5. Bukti Transaksi	Nota bon tulisan tangan yang rentan hilang, robek, atau tintanya luntur seiring waktu.	Pencetakan nota terformat berbasis karakter ASCII yang rapi, memuat nomor transaksi unik, waktu ISO-8601, rincian barang, dan nominal kembalian.
6. Laporan Keuangan	Pemilik toko mengumpulkan tumpukan potongan nota kertas setiap tutup toko untuk menjumlahkan total pendapatan harian secara manual.	Pemilik toko cukup memilih menu laporan untuk melihat ringkasan omzet dan jumlah nota yang berhasil diproses secara otomatis.

3.2 Diagram Alir Transaksi Penjualan (Flowchart Checkout Kasir)

Flowchart berikut merepresentasikan logika algoritma checkout kasir yang menjamin sifat **Atomisitas**: transaksi dinyatakan sukses seutuhnya (nota tercetak dan stok berkurang), atau gagal seutuhnya (tidak ada stok yang

© 2026 Hello Project · Hak Cipta Dilindungi

berkurang dan nota tidak dicetak).



Gambar 3.1: Flowchart Logika Transaksi Checkout KasirKita (Validasi Berlapis, Transaksi Atomik, dan Rollback)

Aturan Emas Transaksi Atomik:
Perhatikan cabang keputusan pada diagram alir di atas:

- **Validasi Stok:** Jika pembeli meminta kuantitas barang melebihi stok yang tercatat pada file, proses langsung diarahkan ke blok kesalahan. Tidak ada data yang diubah!
- **Validasi Pembayaran:** Jika uang tunai yang diserahkan pembeli lebih kecil daripada total belanja (`paid < total`), sistem menolak transaksi.
- **Mekanisme Rollback:** Jika terjadi kegagalan saat menulis ke berkas transaksi (misalnya disk penuh atau file terkunci), seluruh perubahan stok yang sempat dipersiapkan harus dibatalkan (*rollback*). Nota transaksi hanya boleh dicetak setelah seluruh penyimpanan data dinyatakan sukses 100%!

3.3 Pemodelan Konseptual: ERD Awal Berbasis Nota Fisik

Langkah awal merancang basis data adalah membedah dokumen fisik yang sudah digunakan toko, yaitu **Nota Kertas Belanja**. Dari nota tersebut, kita mengidentifikasi entitas awal dan relasinya sebelum dilakukan normalisasi.



Gambar 3.2: Entity-Relationship Diagram (ERD) Awal — Model Belum Normal Berdasarkan Nota Fisik

Kelemahan & Anomali pada ERD Awal:
Model data di atas belum normal karena:

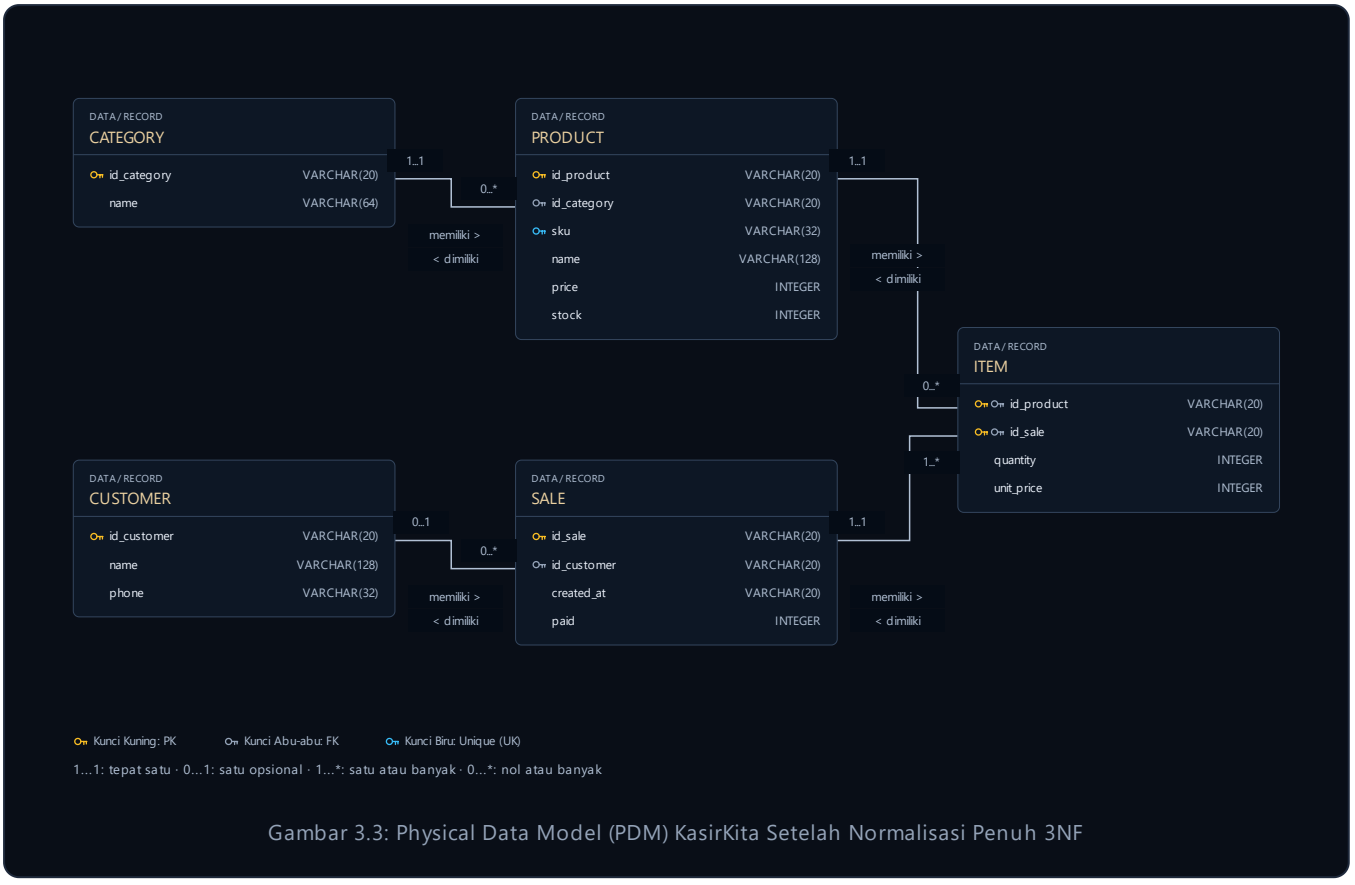
1. **Kelompok Berulang (Repeating Groups):** Entitas `NOTA_AWAL` menyimpan atribut `items[]` sebagai array/daftar barang di dalam satu catatan nota. Hal ini melanggar kaidah relasional dasar karena satu sel memuat banyak nilai.
2. **Redundansi Data:** Nama kategori barang dan nama pelanggan ditulis berulang-ulang di setiap nota, menyebabkan pemborosan ruang simpan dan risiko ketidakkonsistenan saat pengetikan.
3. **Anomali Pembaruan (Update Anomaly):** Jika harga barang berubah, histori nota lama yang menggunakan nama barang tersebut rentan ikut berubah secara keliru jika tidak dipisahkan.

3.4 Normalisasi Basis Data Fisik (PDM: 1NF, 2NF, hingga 3NF)

Untuk mengatasi anomali pada ERD awal, kita menerapkan teknik **Normalisasi** secara berjenjang hingga mencapai **Third Normal Form (3NF)**.

© 2026 Hello Project · Hak Cipta Dilindungi

Tahap Normalisasi	Tindakan Transformasi yang Dilakukan	Tujuan & Manfaat Teknis
Bentuk Tidak Normal (UNF) ke 1NF	Membongkar array `items[]` pada nota menjadi entitas relasional baru bernama `ITEM` (atau `sale_items`). Setiap baris data kini hanya memuat nilai skalar tunggal (*atomic value*).	Menghilangkan kelompok berulang (*repeating groups*). Setiap item barang pada nota kini memiliki baris tersendiri yang dapat di-query secara independen.
1NF ke 2NF	Pada tabel `ITEM`, primary key merupakan kunci gabungan (*Composite Key*: `id_sale` + `id_product`). Atribut seperti nama produk dan harga master dikeluarkan dari `ITEM` dan dipindahkan ke tabel master `PRODUCT`.	Menghilangkan ketergantungan parsial (*partial functional dependency*). Semua atribut non-kunci di tabel item (seperti `quantity` dan `unit_price`) kini bergantung penuh pada seluruh pasangan composite key.
2NF ke 3NF	Memisahkan atribut yang bergantung pada atribut non-kunci lainnya (ketergantungan transitif). Nama kategori dipisahkan ke tabel master `CATEGORY`. Data identitas pembeli dipisahkan ke tabel master `CUSTOMER`.	Menghilangkan ketergantungan transitif (*transitive functional dependency*). Informasi kategori dan pelanggan kini memiliki entitas unik sendiri dan direferensikan via *Foreign Key*.

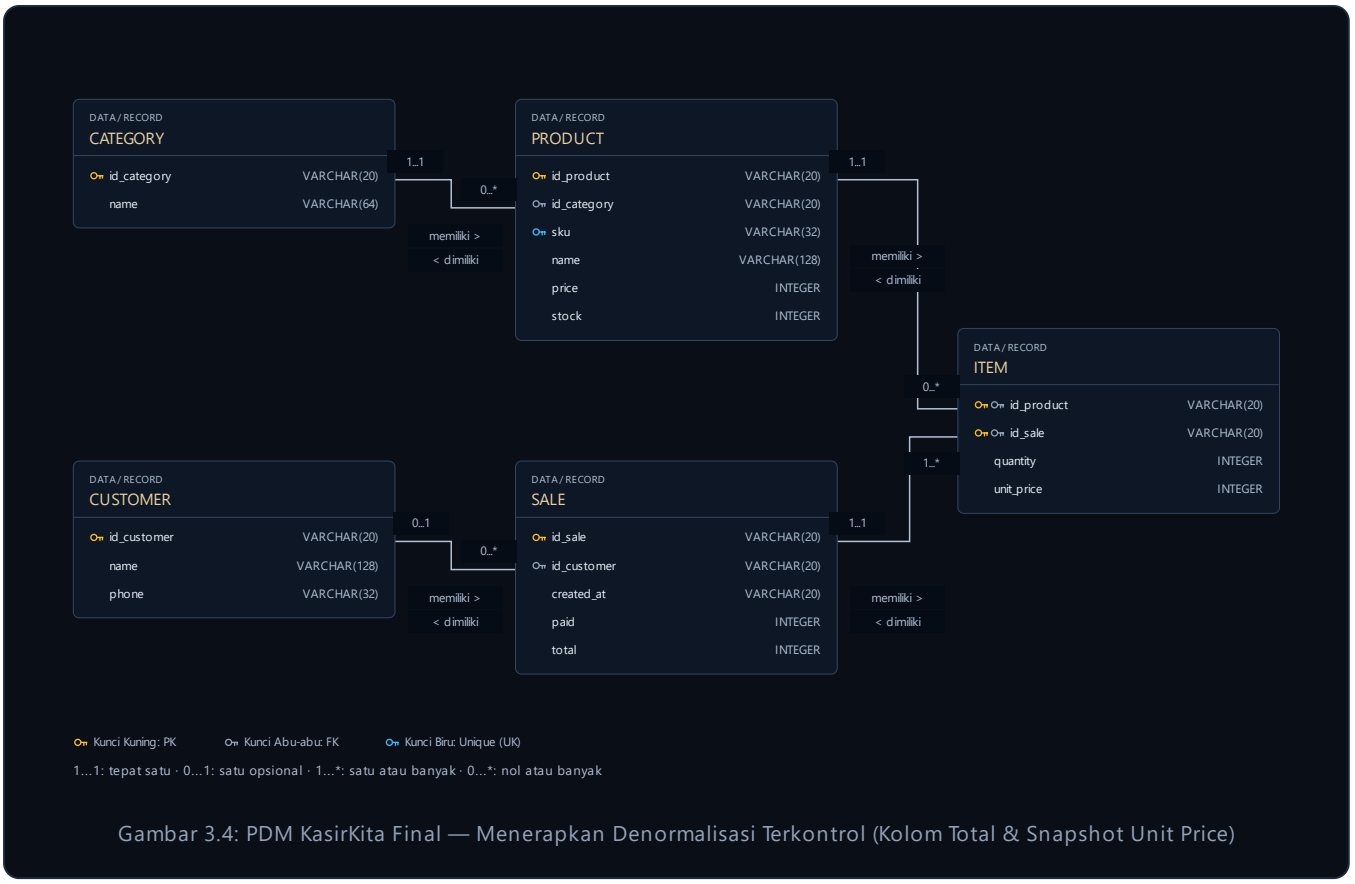


Gambar 3.3: Physical Data Model (PDM) KasirKita Setelah Normalisasi Penuh 3NF

3.5 Denormalisasi Terkontrol: Snapshot Harga & Penyimpanan Total

Meskipun normalisasi 3NF menghasilkan struktur yang bersih secara akademis, pada aplikasi penjualan dunia nyata terdapat dua kebutuhan krusial yang memerlukan **Denormalisasi Terkontrol**:

© 2026 Hello Project · Hak Cipta Dilindungi



Gambar 3.4: PDM KasirKita Final — Menerapkan Denormalisasi Terkontrol (Kolom Total & Snapshot Unit Price)

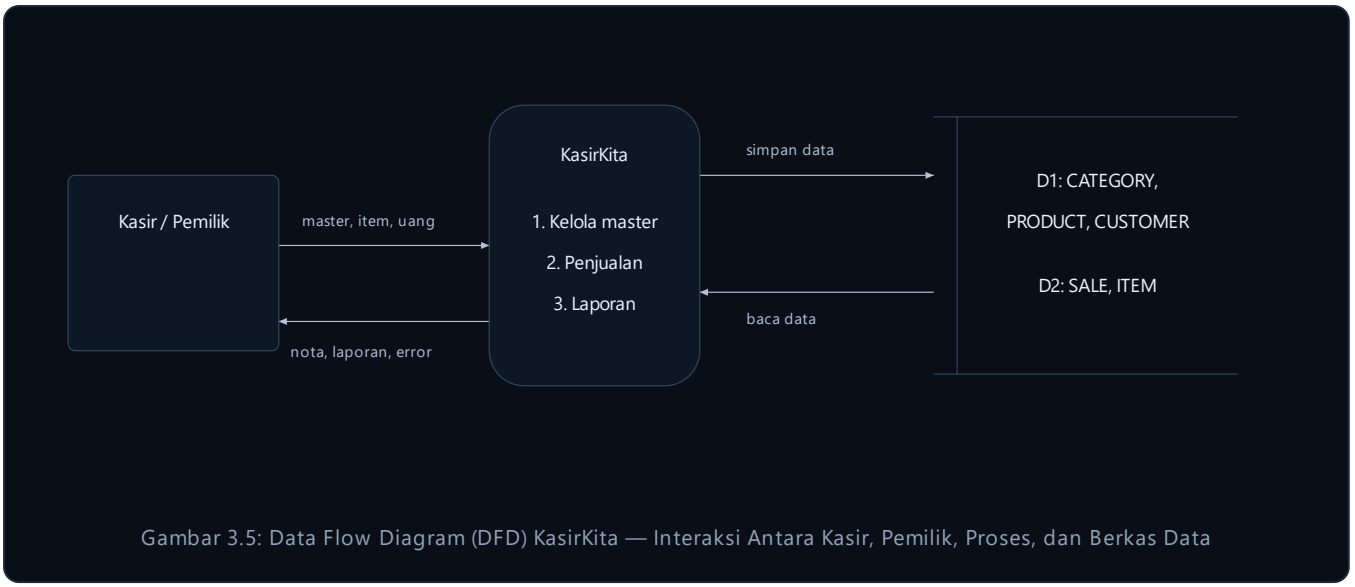
Dua Keputusan Denormalisasi pada KasirKita:

1. **Penambahan Kolom `total` pada Tabel `SALE`:**
   Secara teori relasional murni, total belanja dapat dihitung setiap saat dengan rumus `SUM(quantity × unit_price)`. Namun, untuk mempercepat pencarian riwayat transaksi, pembukaan daftar nota, dan penyusunan laporan harian, nilai total dihitung satu kali di memori dan disimpan secara permanen pada header nota. Uang kembalian tetap dihitung secara dinamis dari rumus `paid - total`.
2. **Penyimpanan Snapshot Harga (`unit_price`) pada `ITEM`:**
   Ini adalah aturan integritas finansial yang paling krusial: tabel `ITEM` wajib menyimpan nilai `unit_price` pada saat transaksi terjadi. Jika minggu depan pemilik toko menaikkan harga Buku Tulis A5 dari Rp12.000 menjadi Rp15.000 pada tabel `PRODUCT`, nota transaksi milik Rani yang terjadi hari ini **TIDAK BOLEH** berubah menjadi Rp15.000! Sejarah audit transaksi keuangan toko harus tetap utuh dan tidak terdistorsi.

3.6 Data Flow Diagram (DFD): Aliran Data Sistem KasirKita

Data Flow Diagram (DFD) menggambarkan bagaimana data mengalir dari entitas eksternal (pengguna), masuk ke dalam proses komputasi, disimpan ke media persisten (*Data Store*), dan dikembalikan sebagai informasi luaran.

© 2026 Hello Project · Hak Cipta Dilindungi



Gambar 3.5: Data Flow Diagram (DFD) KasirKita — Interaksi Antara Kasir, Pemilik, Proses, dan Berkas Data

Tingkatan DFD	Komponen & Entitas	Aliran Masukan (Input)	Proses & Pengolahan Data	Aliran Keluaran (Output)
DFD Level 0 (Diagram Konteks)	• Entitas Eksternal: **Kasir** • Entitas Eksternal: **Pemilik** • Proses Utama: **0. KasirKita**	• Kasir mengirimkan pilihan SKU produk, kuantitas beli, dan uang tunai. • Pemilik mengirimkan data produk baru, pembaruan stok, dan parameter tanggal laporan.	Sistem KasirKita bertindak sebagai kotak hitam tunggal yang memvalidasi otorisasi, mengeksekusi kalkulasi, dan mengelola berkas data.	• Kasir menerima struk nota belanja atau pesan kesalahan validasi. • Pemilik menerima konfirmasi pembaruan master dan rekap omzet penjualan harian.
DFD Level 1 (Dekomposisi Proses)	• **Proses 1.0: Kelola Master** • **Proses 2.0: Proses Penjualan** • **Proses 3.0: Susun Laporan** • Data Store **D1: MASTER** (Kategori, Produk, Pelanggan) • Data Store **D2: TRANSAKSI** (Penjualan, Detail Item)	• Proses 1.0: Menerima input data master dari Pemilik. • Proses 2.0: Menerima input item belanja dan uang dari Kasir; membaca data stok dari D1. • Proses 3.0: Menerima filter rentang tanggal dari Pemilik; membaca arsip dari D2.	• 1.0: Menulis/memperbarui rekaman ke D1. • 2.0: Mengurangi stok pada D1; menulis nota baru dan item ke D2 secara atomik. • 3.0: Menghitung akumulasi total pendapatan dan jumlah nota dari D2 (hanya baca/read-only).	• 1.0: Status sukses simpan master. • 2.0: Lembar nota belanja fisik. • 3.0: Tabel rekap omzet keuangan toko.

3.7 Kamus Data Komprehensif (Data Dictionary)

© 2026 Hello Project · Hak Cipta Dilindungi

Kamus data menetapkan spesifikasi teknis setiap atribut data, tipe data pada bahasa C, batasan nilai (*constraints*), dan contoh representasi nilainya.

© 2026 Hello Project · Hak Cipta Dilindungi

Tabel / Entitas	Nama Atribut (Field)	Tipe Data (C / SQLite)	Aturan & Validasi Teknis	Contoh Nilai
categories (Master Kategori)	`id_category`	INTEGER / VARCHAR(20)	Primary Key; penomoran sekuensial unik; tidak boleh duplikat.	`1` atau `CAT01`
	`name`	TEXT / VARCHAR(64)	NOT NULL; UNIQUE; nama kategori barang toko ATK.	`Alat Tulis`
products (Master Katalog)	`id_product`	INTEGER / VARCHAR(20)	Primary Key; penomoran identitas unik produk.	`1` atau `PRD001`
	`id_category`	INTEGER / VARCHAR(20)	Foreign Key mereferensikan `categories.id_category`; NOT NULL.	`1`
	`sku`	TEXT / VARCHAR(32)	NOT NULL; UNIQUE; kode identifikasi barcode barang.	`P001`
	`name`	TEXT / VARCHAR(128)	NOT NULL; nama deskripsi lengkap barang.	`Buku Tulis A5`
	`price`	INTEGER	NOT NULL; CHECK (price >= 0); harga satuan rupiah bulat.	`12000`
	`stock`	INTEGER	NOT NULL; CHECK (stock >= 0); jumlah fisik tersedia di toko.	`20`
customers (Master Pelanggan)	`id_customer`	INTEGER / VARCHAR(20)	Primary Key; identitas pembeli terdaftar.	`1`
	`name`	TEXT / VARCHAR(128)	NOT NULL; nama pembeli (default jika umum: "Pelanggan Umum").	`Rani`
	`phone`	TEXT / VARCHAR(32)	NULLABLE; nomor telepon kontak pembeli.	`081234567890`
sales (Header Penjualan)	`id_sale`	INTEGER / VARCHAR(20)	Primary Key; nomor nota unik transaksi belanja.	`1` atau `NOT-0001`
	`id_customer`	INTEGER / VARCHAR(20)	Foreign Key ke `customers.id_customer`; boleh NULL jika pembeli umum.	`1`
	`created_at`	TEXT / VARCHAR(32)	NOT NULL; format tanggal & waktu ISO-8601 UTC.	`2026-10-20T14:30:00Z`
	`total`	INTEGER	NOT NULL; CHECK (total >= 0); denormalisasi total belanja.	`24000`
	`paid`	INTEGER	NOT NULL; CHECK (paid >= total); nominal uang tunai yang diserahkan.	`30000`

© 2026 Hello Project · Hak Cipta Dilindungi

Tabel / Entitas	Nama Atribut (Field)	Tipe Data (C / SQLite)	Aturan & Validasi Teknis	Contoh Nilai
sale_items (Detail Penjualan)	`id_sale`	INTEGER / VARCHAR(20)	Composite Primary Key; Foreign Key ke `sales.id_sale`.	`1`
	`id_product`	INTEGER / VARCHAR(20)	Composite Primary Key; Foreign Key ke `products.id_product`.	`1`
	`quantity`	INTEGER	NOT NULL; CHECK (quantity > 0); jumlah unit barang yang dibeli.	`2`
	`unit_price`	INTEGER	NOT NULL; CHECK (unit_price >= 0); snapshot harga satuan saat transaksi.	`12000`

3.8 Desain Penyimpanan Berkas Teks (.txt) pada Bahasa C

Karena proyek KasirKita menggunakan teknik *file processing* tanpa engine database SQL pihak ketiga, data disimpan ke dalam berkas teks terstruktur pada folder data/ dengan pemisah karakter pipa (|) atau baris per baris.

data/produk.txt · Format Penyimpanan Katalog Master — File Format

```
# ID_PRODUK|ID_KATEGORI|SKU|NAMA_PRODUK|HARGA|STOK
1|1|P001|Buku Tulis A5|12000|20
2|1|P002|Pulpen Gel 0.5 Hitam|5000|50
3|2|P003|Kertas HVS A4 80gr 1 Rim|48000|15
4|1|P004|Penghapus Pensil 4B|3000|40
```

data/penjualan.txt · Format Penyimpanan Header Transaksi — File Format

```
# ID_NOTA|ID_PELANGGAN|WAKTU_ISO|TOTAL|DIBAYAR
1|1|2026-10-20T10:15:00|24000|30000
2|0|2026-10-20T11:40:00|5000|10000
```

data/penjualan_detail.txt · Format Penyimpanan Rincian Item Transaksi — File Format

```
# ID_NOTA|ID_PRODUK|QTY|HARGA_SATUAN_SNAPSHOT
1|1|2|12000
2|2|1|5000
```

3.9 Matriks Fitur & Kriteria Keberhasilan (Acceptance Criteria)

© 2026 Hello Project · Hak Cipta Dilindungi

Modul / Fitur	Kriteria Keberhasilan Fungsional (Penerimaan Sistem)
Pencarian Katalog	Kasir dapat mencari barang berdasarkan kode SKU atau penggalan nama produk. Menampilkan harga dan sisa stok fisik secara akurat.
Kelola Keranjang	Dapat menambah item, mengubah kuantitas beli, atau membatalkan item dari keranjang. Menolak kuantitas bernilai nol atau negatif.
Validasi Checkout	Memastikan stok barang mencukupi sebelum uang diterima. Menolak transaksi jika uang tunai pembeli kurang dari total tagihan.
Pencatatan Berkas	Menulis transaksi ke `penjualan.txt` dan memperbarui sisa stok di `produk.txt` dalam satu kesatuan operasi yang konsisten.
Pencetakan Struk Nota	Menampilkan lembar nota terminal yang memuat identitas toko, nomor nota unik, tanggal, rincian barang, total, bayar, dan uang kembalian.
Laporan Penjualan	Mampu menghitung total omzet pendapatan toko dan total kuantitas nota yang berhasil dicetak pada tanggal tertentu secara cepat.

© 2026 Hello Project · Hak Cipta Dilindungi

MODUL 02: DOKUMEN AFTER

BAB 4: Dokumen After — Pengujian & Serah Terima

4.1 Metodologi User Acceptance Testing (UAT) Kasir

User Acceptance Testing (UAT) adalah tahapan krusial di mana calon pengguna nyata—yaitu kasir dan pemilik toko alat tulis—mengoperasikan langsung perangkat lunak untuk memastikan bahwa sistem benar-benar memenuhi kebutuhan operasional dan bebas dari cacat logika kritis.

Pengujian perangkat lunak rekayasa tidak boleh sekadar menguji "apakah halaman atau menu bisa terbuka". Fokus utama pengujian KasirKita adalah **Integritas Finansial, Ketepatan Stok Fisik, dan Ketahanan Transaksi Atomik**. Hasil yang diharapkan (*expected result*) wajib didefinisikan secara tertulis sebelum pengujian dieksekusi.

Uji Kasus Positif (Positive Testing)

Menguji alur belanja wajar: kasir memasukkan barang yang stoknya tersedia, pembeli menyerahkan uang tunai pas atau lebih, nota tercetak dengan benar, dan kembalian terhitung presisi.

Uji Kasus Negatif & Batas (Negative & Boundary Testing)

Menguji ketahanan sistem saat disuntikkan data salah: memasukkan huruf pada kolom angka, kuantitas belanja melebihi stok, uang tunai kurang, pembatalan tiba-tiba, dan karakter khusus.

4.2 Matriks Skenario UAT KasirKita

Tabel berikut adalah matriks pengujian komprehensif KasirKita yang mencakup 12 skenario pengujian utama:

© 2026 Hello Project · Hak Cipta Dilindungi

ID / Level Uji	Skenario Pengujian & Langkah Operasi	Hasil yang Diharapkan (Expected Result)	Kriteria Verifikasi Audit Data
T01 · E2E (Normal)	Reset stok Buku A5 = 20. Beli 2 buku @ Rp12.000. Serahkan uang tunai Rp30.000.	Total tagihan Rp24.000; kembalian Rp6.000; nota belanja tercetak rapi; stok buku di file berkurang menjadi 18 unit.	1 record baru di `penjualan.txt`, 1 detail di `penjualan_detail.txt`, stok di `produk.txt` = 18.
T02 · Validasi Stok	Stok Pulpen = 1. Kasir mencoba memasukkan kuantitas beli = 2.	Sistem menolak dengan pesan jelas: *"Stok tidak mencukupi (sisa: 1)"*. Transaksi tidak diproses.	Stok pulpen di berkas tetap 1; tidak ada record baru yang ditambahkan ke berkas penjualan.
T03 · Validasi Tipe Data	Pada input kuantitas barang, kasir mengetikkan angka `0`, negatif `-5`, desimal `2.5`, atau huruf `abc`.	Sistem menolak masukan secara seketika; kursor tetap bertahan pada form meminta angka bilangan bulat positif (> 0).	Berkas data master dan transaksi sama sekali tidak mengalami perubahan.
T04 · Pembayaran Kurang	Total belanja belanja Rp24.000. Kasir memasukkan uang tunai yang diserahkan pembeli = Rp20.000.	Sistem menolak pembayaran dengan pesan: *"Uang pembayaran kurang Rp4.000"*. Nota belanja TIDAK dicetak.	Stok barang tidak berkurang; berkas transaksi tidak bertambah.
T05 · Uji Atomisitas	Simulasikan kegagalan penulisan detail belanja ke disket/folder (misal file `penjualan_detail.txt` di-lock atribut read-only).	Sistem mendeteksi kegagalan I/O, menampilkan pesan error, dan membatalkan (rollback) seluruh transaksi.	Header penjualan dibatalkan dan pengurangan stok dikembalikan ke posisi semula. Tidak ada data korup!
T06 · Batas Stok Habis	Stok tersisa 1 unit. Pembeli membeli tepat 1 unit dan membayar lunas.	Transaksi sukses; total terhitung benar; nota tercetak; stok di berkas katalog kini menjadi tepat `0` (bukan negatif).	Transaksi berikutnya atas barang ini otomatis ditolak karena stok = 0.
T07 · Regresi Harga	Setelah transaksi T01 selesai, pemilik menaikkan harga Buku A5 di katalog dari Rp12.000 menjadi Rp15.000. Buka riwayat nota T01.	Nota T01 tetap menampilkan harga satuan Rp12.000 dan total Rp24.000 (tidak terpengaruh kenaikan harga katalog).	Nilai `unit_price` pada arsip historis terlindungi berkat denormalisasi snapshot harga.
T08 · Validasi Master	Pemilik toko mencoba mendaftarkan produk baru dengan kode SKU yang sudah ada (duplikat), atau harga jual negatif.	Sistem menolak penyimpanan dengan peringatan kode SKU sudah terdaftar atau nilai harga tidak valid.	File `produk.txt` menolak rekaman ganda.

© 2026 Hello Project · Hak Cipta Dilindungi

ID / Level Uji	Skenario Pengujian & Langkah Operasi	Hasil yang Diharapkan (Expected Result)	Kriteria Verifikasi Audit Data
T09 · Laporan Keuangan	Jalankan transaksi pada tanggal 20 Oktober pukul 23:59 dan 21 Oktober pukul 00:05. Cetak laporan tanggal 20 Oktober.	Hanya transaksi pukul 23:59 yang masuk ke omzet tanggal 20. Transaksi pukul 00:05 masuk ke omzet tanggal 21.	Akumulasi omzet dan jumlah nota terpisah akurat berdasarkan batas waktu kalender lokal.
T10 · Responsivitas UI	Jalankan program pada jendela console ukuran standar (80 kolom × 25 baris). Uji navigasi panah keyboard dan enter.	Antarmuka tidak berantakan (*no line wrapping*); teks berada rapi di dalam kotak bingkai; kursor berpindah mulus.	Estetika visual konsisten tanpa glitch teks.
T11 · Keamanan Total	Simulasikan injeksi data dengan mencoba mengirimkan total bayar yang dimanipulasi dari luar form.	Sistem mengabaikan masukan total eksternal dan selalu menghitung ulang total dari perkalian kuantitas dengan harga resmi berkas.	Kalkulasi server/backend menjadi otoritas tunggal kebenaran nilai uang.
T12 · Penggabungan Keranjang	Kasir memilih Buku A5 sebanyak 1 buah, lalu beberapa saat kemudian memilih Buku A5 lagi sebanyak 2 buah pada nota yang sama.	Sistem menggabungkan item tersebut menjadi satu baris: Buku A5 qty = 3, subtotal = Rp36.000.	Nota belanja ringkas tanpa pengulangan baris item yang identik.

4.3 Pelacakan Cacat (Bug Hunter & Defect Management)

Ketika hasil aktual yang diperoleh saat pengujian tidak sesuai dengan hasil yang diharapkan, temuan tersebut diklasifikasikan sebagai **Defect (Cacat / Bug)** dan dicatat ke dalam log pelacakan bug dengan format baku:

© 2026 Hello Project · Hak Cipta Dilindungi

docs/defect-log.txt · Template Laporan Bug KasirKita Format Standar

```
========================================================================
LAPORAN CACAT PERANGKAT LUNAK (BUG REPORT)
========================================================================
ID Cacat        : BUG-001
Terkait Kasus Uji: T04 (Pembayaran Kurang)
Tingkat Keparahan: KRITIS (Blocker - Masalah Keuangan)
Penemu Cacat    : Kasir Toko / QA Tester
Tanggal Ditemukan: 2026-10-21
Status          : DALAM PERBAIKAN

DESKRIPSI:
Saat total belanja Rp24.000 dan kasir menginput uang bayar Rp20.000,
program mencetak nota belanja dengan nilai kembalian negatif (-Rp4.000),
dan stok barang di produk.txt tetap berkurang.

LANGKAH REPRODUKSI:
1. Buka menu Transaksi Penjualan.
2. Tambahkan 2 Buku A5 (Total = Rp24.000).
3. Pada input "Uang Dibayar", ketikkan 20000 lalu tekan Enter.
4. Perhatikan layar nota dan periksa file data/produk.txt.

HASIL AKTUAL    : Nota tercetak dengan kembalian -4000; stok berkurang.
HASIL EKSPEKTASI: Sistem menampilkan pesan "Uang pembayaran kurang Rp4.000",
                  menolak mencetak nota, dan stok barang TIDAK berkurang.

TINDAKAN PERBAIKAN:
Tambahkan kondisi validasi pada fungsi proses_checkout():
if (uang_dibayar < total_belanja) {
    tampilkan_error("Uang kurang!");
    return BATAL;
}
========================================================================
```

4.4 Panduan Penyusunan Proposal Proyek Perangkat Lunak

Proposal proyek diajukan di awal inisiasi kerja sama tim dengan dosen pengampu atau pemilik usaha toko ATK. Proposal yang baik memuat argumen bisnis yang meyakinkan serta rencana teknis yang realistis.

© 2026 Hello Project · Hak Cipta Dilindungi

Struktur Proposal	Uraian Isi yang Wajib Dilengkapi
1. Halaman Judul & Identitas	Judul proyek, nama tim pengembang, nomor induk mahasiswa/kepegawaian, tanggal pengajuan, dan versi dokumen.
2. Latar Belakang & Masalah	Uraian kondisi nyata toko ATK, kesulitan pencatatan nota manual, potensi kerugian finansial akibat selisih stok, dan urgensi komputerisasi kasir.
3. Tujuan & Manfaat	Tujuan spesifik yang dapat diukur (misal: memproses transaksi di bawah 30 detik, menghilangkan selisih stok, menyediakan laporan omzet otomatis).
4. Ruang Lingkup Sistem (Scope)	Penjelasan tegas mengenai fitur yang dikerjakan (*in-scope*) dan fitur yang ditunda ke versi lanjutan (*out-of-scope*) untuk mengunci ekspektasi.
5. Rencana Kerja (Activity Plan)	Tabel rincian aktivitas, pembagian tugas spesifik antar-anggota kelompok, jadwal mingguan, dan bukti serah terima (deliverable).
6. Kebutuhan Sumber Daya	Kebutuhan perangkat keras minimal komputer kasir, sistem operasi Windows, compiler MinGW-w64, dan media pencadangan data.

4.5 Product Documentation (Spesifikasi Teknis & Database)

Product Documentation adalah dokumen rujukan utama bagi pengembang perangkat lunak, dosen penguji, maupun tim pemeliharaan sistem di masa depan. Dokumen ini membedah arsitektur teknis dari dalam.

Sistematika Dokumen Product Documentation:

1. **Product Identification & Revision History:** Mencatat judul produk, pemilik produk, tanggal rilis, nomor revisi, dan riwayat modifikasi.
2. **Introduction:** Deskripsi umum sistem, tujuan bisnis, pemangku kepentingan (*stakeholders*), dan skenario pemakaian tipikal.
3. **Development Process:** Pemodelan proses bisnis *As-Is* dan *To-Be*, diagram alir *flowchart checkout* atomik, serta justifikasi keputusan desain.
4. **System Architecture:** Arsitektur 3 lapisan (Antarmuka Console, Logika Bisnis C, Penyimpanan Berkas), struktur direktori proyek, dan daftar dependensi pustaka.
5. **Data Design:** Skema ERD konseptual, PDM ternormalisasi 3NF, aturan denormalisasi, kamus data lengkap setiap field, dan format penyimpanan file teks `.txt`.
6. **Testing & Acceptance Summary:** Rangkuman hasil pengujian 12 skenario UAT, persentase keberhasilan kasus uji, log penanganan bug, dan bukti audit integritas data.

4.6 User Guide / User Manual KasirKita

User Guide (Buku Petunjuk Operasional) ditujukan khusus untuk pengguna awam (kasir toko) yang tidak memiliki latar belakang teknis pemrograman. Petunjuk harus ditulis dalam bahasa yang lugas, komunikatif, dan berorientasi pada tugas nyata.

© 2026 Hello Project · Hak Cipta Dilindungi

Bagian User Manual	Panduan Operasional untuk Kasir
1. Persyaratan Sistem	Komputer/Laptop dengan OS Windows 10/11; keyboard fisik; ruang disket kosong minimal 50 MB; resolusi layar minimal 1024×768. Sistem beroperasi offline tanpa perlu internet.
2. Menjalankan Aplikasi	Buka folder `KasirKita`, klik ganda pada file `kasirkita.exe`, atau buka melalui terminal PowerShell dengan mengetikkan `.\kasirkita.exe`.
3. Navigasi Menu Utama	Gunakan tombol panah **[▲ / ▼]** pada keyboard untuk memilih menu, lalu tekan **[ENTER]** untuk masuk. Tekan **[ESC]** kapan saja untuk kembali ke menu sebelumnya.
4. Melayani Transaksi Belanja	1. Pilih menu *[1] Transaksi Penjualan*. 2. Ketikkan kode SKU atau nama barang, lalu tekan Enter. 3. Masukkan jumlah (kuantitas) barang yang dibeli. 4. Ulangi langkah untuk barang berikutnya. Tekan [F5] jika belanja selesai. 5. Ketikkan jumlah uang tunai yang diserahkan pembeli. 6. Sistem menampilkan uang kembalian dan mencetak struk nota belanja secara otomatis.
5. Menambah & Mengubah Stok	Pilih menu *[2] Kelola Produk*. Pemilik toko dapat memasukkan data barang baru atau menambah stok barang yang baru saja tiba dari pemasok.
6. Mencetak Laporan Harian	Pilih menu *[3] Laporan Keuangan*. Masukkan tanggal yang diinginkan. Sistem langsung merangkum total uang yang masuk ke laci kasir dan total transaksi.
7. Mengatasi Masalah (Troubleshooting)	• *Kursor hilang atau karakter berantakan:* Tekan tombol [R] untuk me-refresh layar. • *File data tidak bisa disimpan:* Pastikan file `produk.txt` tidak sedang dibuka di aplikasi Notepad atau Microsoft Excel saat transaksi berlangsung.

4.7 Format Laporan Akhir & Checklist Serah Terima

Checklist Verifikasi Dokumen Akhir Proyek:

- ✓ Seluruh 12 skenario kasus uji kritis (keuangan, stok, dan rollback) telah dieksekusi dengan status LULUS.
- ✓ Log cacat (defect log) telah mencatat seluruh perbaikan bug hingga tuntas tanpa ada bug kritis yang dibiarkan terbuka.
- ✓ Nama tabel, nama field pada kamus data, diagram ERD/PDM, dan kode program telah dicocokkan 100% konsisten.
- ✓ Product Documentation dan User Guide siap dicetak atau diserahkan dalam bentuk PDF rapi.
- ✓ Program dapat dijalankan langsung pada laptop baru hanya dengan mengikuti petunjuk pada User Guide.

© 2026 Hello Project · Hak Cipta Dilindungi

MODUL 03: PEMROGRAMAN C

BAB 5: Pemrograman Console Windows dengan lib.h

5.1 Arsitektur Antarmuka Console Windows API & Peran lib.h

Secara default, aplikasi berbasis console terminal di bahasa C hanya menyediakan mekanisme input/output aliran sekuensial sederhana seperti `printf()` dan `scanf()`. Keluaran teks akan terus mengalir ke bawah dan kursor berpindah mengikuti baris baru, tanpa kemampuan mengatur letak kursor secara bebas, menggambar bingkai kotak, menyembunyikan kursor saat menu aktif, maupun menyamarkan kata sandi dengan bintang.

Sistem operasi Microsoft Windows menyediakan antarmuka tingkat rendah bernama **Windows Console API** (terdefinisi pada `<windows.h>`). Pustaka **`lib.h`** adalah modul pembungkus (*wrapper header*) yang mengemas pemanggilan fungsi-fungsi rumit Windows API tersebut menjadi sekumpulan fungsi C yang ringkas, elegan, dan siap pakai untuk membangun antarmuka terminal bergaya retro (*Text-based User Interface - TUI*).

> **Karakteristik Teknis `lib.h`:**
> Pustaka `lib.h` yang digunakan dalam kurikulum KasirKita berisi *definisi implementasi fungsi* secara langsung. Oleh karena itu, pada tahap latihan awal, header ini di-include satu kali pada berkas utama (`main.c`). Untuk proyek berskala besar yang terdiri atas banyak berkas kompilasi terpisah, deklarasi fungsi (*header prototype*) harus dipisahkan dari implementasi fungsinya untuk menghindari galat *duplicate symbol definition* saat linking.

5.2 Standar Struktur Folder Proyek C di IntelliJ IDEA

Untuk menjaga keteraturan kode dan memudahkan kerja sama kelompok, proyek KasirKita disusun mengikuti struktur direktori baku rekayasa perangkat lunak:

© 2026 Hello Project · Hak Cipta Dilindungi

Struktur Direktori Proyek KasirKita — Folder Hierarchy

```
kasirkita/
├── include/
│   └── lib.h              # Header pustaka manipulasi console Windows
├── src/
│   ├── main.c             # Titik masuk utama program & inisialisasi console
│   ├── menu.c / menu.h    # Logika navigasi antarmuka dan penanganan tombol
│   ├── transaksi.c / .h   # Logika proses checkout kasir, validasi stok & uang
│   └── storage.c / .h     # Operasi pembacaan & penulisan berkas teks .txt
├── data/
│   ├── produk.txt         # Berkas penyimpan data master katalog & stok
│   ├── kategori.txt       # Berkas penyimpan data kategori barang
│   ├── penjualan.txt      # Berkas penyimpan header transaksi nota
│   └── penjualan_detail.txt # Berkas penyimpan rincian item transaksi
├── assets/
│   └── ascii/
│       ├── header_toko.txt # Banner ASCII art nama toko
│       └── struk_logo.txt  # Logo kecil ASCII art untuk kepala nota
├── build/
│   └── kasirkita.exe      # Berkas binary executable hasil kompilasi GCC
├── docs/                  # Seluruh dokumen AP, Probis, UAT, dan Manual
└── .gitignore             # Konfigurasi filter berkas pelacak Git
```

5.3 Persiapan Compiler MinGW-w64 & Konfigurasi Standar C17

Program KasirKita dikembangkan menggunakan standar bahasa pemrograman **C17** dengan kompiler **GCC (MinGW-w64)** di lingkungan Windows. Karena `lib.h` memanfaatkan fungsi Windows API dan manipulasi karakter non-buffer (`_getch()`), berkas C wajib mengikutsertakan header standar pendukung sebelum memanggil `lib.h`:

src/main.c · Kerangka Inklusi Header Awal — Bahasa C17

```c
#include <stdio.h>      // Operasi I/O standar (printf, fopen, fgets)
#include <stdlib.h>     // Alokasi memori, exit, system
#include <windows.h>    // Windows Console API (COORD, HANDLE, SetConsoleCursorPosition)
#include <conio.h>      // Input karakter langsung tanpa enter (_getch)
#include <wchar.h>      // Manipulasi string lebar untuk setConsoleFont
#include <ctype.h>      // Pengecekan karakter (isdigit, isalpha)
#include "lib.h"        // Pustaka antarmuka console KasirKita

int main(void) {
    // 1. Inisialisasi jendela terminal
    remove_scrollbar();
    hideCursor();

    // 2. Menampilkan teks selamat datang
    gotoPrintString(10, 5, "Halo! KasirKita siap beroperasi.");

    // 3. Menunggu tombol sembarang sebelum keluar
    _getch();
    showCursor();
    return 0;
}
```

© 2026 Hello Project · Hak Cipta Dilindungi

Terminal PowerShell · Perintah Kompilasi & Eksekusi CLI

```
# Berpindah ke direktori utama proyek
cd kasirkita

# Membuat direktori output binary jika belum ada
mkdir -p build

# Mengompilasi kode C dengan peringatan penuh (-Wall -Wextra)
gcc -std=c17 -Wall -Wextra src/main.c -Iinclude -o build/kasirkita.exe

# Menjalankan program kasir
.\build\kasirkita.exe
```

5.4 Sistem Koordinat Console Windows (Sumbu X & Y, Grid 80×25)

Console terminal Windows memetakan layar monitor sebagai kisi-kisi (*grid*) karakter dua dimensi dengan sistem koordinat Cartesian `(x, y)`:

- **Titik Asal (Origin `0, 0`):** Terletak persis di pojok kiri paling atas layar terminal.
- **Sumbu X (Horizontal):** Menunjukkan nomor kolom karakter, bertambah besar ke arah **kanan** (rentang standar: 0 sampai 79).
- **Sumbu Y (Vertikal):** Menunjukkan nomor baris karakter, bertambah besar ke arah **bawah** (rentang standar: 0 sampai 24).

> **Penting Mengenai Resolusi Layar:**
> Ukuran standar jendela console Windows yang optimal untuk KasirKita adalah **80 kolom × 25 baris**. Selalu pastikan setiap teks, kotak bingkai, dan formulir input ditempatkan pada rentang koordinat yang aman. Jika sebuah teks dicetak pada `x = 75` dengan panjang 20 karakter, teks tersebut akan menabrak batas layar kanan dan terpotong atau membungkus ke baris berikutnya, merusak tata letak visual!

5.5 Pustaka Fungsi Posisi & Manipulasi Teks

Tiga fungsi utama untuk menempatkan teks secara tepat pada koordinat layar:

- `gotoxy(int x, int y)`: Memindahkan posisi kursor aktif terminal ke kolom x dan baris y menggunakan fungsi Windows API `SetConsoleCursorPosition`.
- `gotoPrintString(int x, int y, char *text)`: Mengombinasikan pemindahan kursor ke `(x, y)` dan langsung mencetak string teks pada posisi tersebut.
- `gotoPrintChar(int x, int y, char ch)`: Memindahkan kursor ke `(x, y)` dan mencetak satu karakter tunggal (sangat berguna untuk menggambar elemen grafis atau penunjuk panah menu >).

5.6 Pustaka Fungsi Bingkai & Pembersihan Area

Untuk membangun antarmuka kasir yang rapi dan profesional seperti aplikasi kasir bank atau minimarket zaman dulu, `lib.h` menyediakan fungsi penggambaran kotak:

© 2026 Hello Project · Hak Cipta Dilindungi

- `borderLine(int x, int y, int width, int height)`: Menggambar bingkai persegi panjang garis tunggal (*single-line box*) mulai dari sudut kiri-atas `(x, y)` hingga `(x + width, y + height)` menggunakan karakter ASCII seperti `-`, `|`, dan `+`.
- `borderDoubleLine(int x, int y, int width, int height)`: Menggambar bingkai persegi panjang bergaris ganda (*double-line box*) yang memberikan aksen visual lebih tegas dan elegan pada menu utama atau judul toko.
- `borderClear(int x, int y, int width, int height)`: Menghapus garis tepi bingkai yang telah digambar dengan menggantinya menjadi spasi kosong.
- `clearArea(int x, int y, int width, int height)`: Mengosongkan seluruh area di dalam kotak (membersihkan baris dan kolom) dengan mencetak karakter spasi. Sangat krusial digunakan saat ingin memperbarui isi formulir kasir tanpa menghapus seluruh layar (*no flicker*).

5.7 Manajemen Visibilitas Kursor & Jendela

- `hideCursor()`: Menyembunyikan kursor kedip terminal saat sistem sedang menampilkan menu pilihan agar tampilan layar terlihat bersih dan tenang.
- `showCursor()`: Menampilkan kembali kursor kedip saat kasir perlu mengetikkan data pada kolom input formulir.
- `fullscreen()`: Memperbesar jendela terminal memenuhi seluruh layar monitor untuk pengalaman kasir POS penuh.
- `remove_scrollbar()`: Menghilangkan bilah gulung vertikal/horizontal terminal agar tampilan tetap statis dan tidak bergeser secara tidak disengaja.
- `setConsoleFont(const wchar_t *fontName, int sizeX, int sizeY)`: Mengubah jenis font terminal (misalnya Consolas atau Lucida Console) dan ukurannya melalui Windows API `SetCurrentConsoleFontEx`.

5.8 Pembacaan File Teks Eksternal: PrintTxt

Fungsi `PrintTxt(char *filePath, int x, int y)` membaca isi berkas teks baris demi baris (misalnya gambar ASCII art toko) dan langsung menampilkannya pada koordinat terminal mulai dari kolom `x` dan baris `y`. Hal ini memungkinkan kita memisahkan desain seni logo dari logika kode pemrograman C.

5.9 Pustaka Fungsi Input Interaktif Khusus

Fungsi input bawaan C seperti `scanf()` memiliki banyak celah fatal saat digunakan pada antarmuka kasir (misalnya program macet jika kasir mengetik huruf pada kolom angka). Pustaka `lib.h` memecahkan masalah ini dengan menyediakan fungsi input interaktif berbasis penangkapan tombol karakter demi karakter:

© 2026 Hello Project · Hak Cipta Dilindungi

Nama Fungsi Input	Argumen & Parameter	Perilaku Khusus & Validasi
`scanInputString()`	`(int x, int y, char *text, int maxChar)`	Menerima masukan teks umum (nama produk, SKU). Menolak tombol panah dan membatasi panjang input maksimal sebesar `maxChar`.
`scanInputText()`	`(int x, int y, char *text, int maxChar)`	Mirip dengan string namun dioptimalkan untuk masukan teks deskripsi atau nama pelanggan yang memperbolehkan spasi.
`scanInputPassword()`	`(int x, int y, char *text, int maxChar)`	Menangkap pengetikan karakter secara rahasia dan menampilkan karakter bintang (*) pada layar terminal untuk keamanan.
`scanInputNumber()`	`(int x, int y, int *num, int maxChar)`	**Hanya menerima karakter angka (0-9).** Tombol huruf diabaikan sepenuhnya sehingga kasir mustahil salah ketik huruf pada kuantitas! Mengonversi masukan menjadi bilangan bulat `int`.
`scanInputMoney()`	`(int x, int y, int *num, int minChar, int maxChar)`	Menerima masukan nominal uang rupiah dengan batasan jumlah digit minimal dan maksimal.

5.10 Analisis Masalah Kritis & Solusi Bug Fixes pada lib.h Asli

Pustaka `lib.h` asli yang umum beredar pada tugas mahasiswa mengandung beberapa **cacat logika (bug) kritis** yang dapat menyebabkan galat kompilasi atau perilaku *undefined behavior*. Sebelum digunakan dalam proyek KasirKita, tim wajib menerapkan koreksi perbaikan berikut pada berkas `include/lib.h`:

© 2026 Hello Project · Hak Cipta Dilindungi

Lokasi / Fungsi	Cacat Logika (Bug) pada File Asli	Dampak Galat	Solusi Perbaikan (Patch)
Semua fungsi input saat tombol BACKSPACE	Kode asli menuliskan: `text[i] = "";`	**Galat Tipe C:** String literal `""` adalah pointer bertipe `char*`, sedangkan elemen array `text[i]` bertipe skalar `char`. Kompiler C17 modern akan menolak baris ini atau memicu peringatan berbahaya.	Kurangi indeks `i` lalu tuliskan karakter null terminator: `--i; text[i] = '\0'; printf("\b \b");`
scanInputString saat tombol ESC	Kondisi pembatalan: `if (i < 0) return;`	**Logika Macet:** Variabel `i` selalu bernilai >= 0 sehingga kondisi tersebut tidak akan pernah terpenuhi. Penekanan tombol ESC tidak membatalkan form secara bersih.	Tetapkan perilaku pembatalan eksplisit: kosongkan buffer `text[0] = '\0'` lalu keluar dari fungsi dengan nilai status batal.
Fungsi PrintTxt	Langsung memanggil `fgets()` setelah `fopen()` tanpa memeriksa apakah pointer berkas valid.	**Crash Fatal (Segmentation Fault):** Jika nama file salah ketik atau file tidak ada di disket, program langsung *crash* seketika.	Wajib menambahkan pengecekan keamanan: `FILE *fp = fopen(filePath, "r"); if (fp == NULL) return;`
Batas Kapasitas Buffer	Parameter `maxChar` tidak memperhitungkan tempat untuk karakter penutup string NUL (`\0`).	**Buffer Overflow:** Jika pengguna mengetik hingga karakter maksimal, terminator NUL terlewat dan string meluap ke memori variabel lain.	Selalu pastikan alokasi array penampung memiliki ukuran minimal `maxChar + 1` dan akhiri selalu dengan `text[i] = '\0'`.
setConsoleFont	Menggunakan fungsi `wcscpy()` tanpa memvalidasi panjang nama font terhadap kapasitas struktur `FaceName`.	Potensi luapan buffer memori Windows API jika nama font terlalu panjang.	Gunakan `wcsncpy()` dengan batasan `LF_FACESIZE - 1`.

© 2026 Hello Project · Hak Cipta Dilindungi

DOKUMENTASI LENGKAP API LIB.H

BAB 6: Referensi Lengkap API lib.h (18 Fungsi)

6.1 Dokumentasi Detail Seluruh Fungsi Console Windows

Bagian ini menyajikan spesifikasi teknis lengkap dari seluruh 18 fungsi yang tersedia pada pustaka `lib.h`, mencakup tanda tangan fungsi (*signature*), daftar parameter, nilai kembalian, perilaku di layar terminal, contoh kode program C nyata, serta catatan implementasi teknis.

1. gotoxy()

Signature Bahasa C — lib.h

```
void gotoxy(int x, int y);
```

Deskripsi: Memindahkan posisi kursor aktif terminal ke koordinat kolom x dan baris y.

Parameter	Tipe Data	Deskripsi
x	int	Nomor kolom karakter (sumbu horizontal, mulai dari 0 di kiri).
y	int	Nomor baris karakter (sumbu vertikal, mulai dari 0 di atas).

Nilai Kembalian: `void` (tidak mengembalikan nilai).

Perilaku: Menggunakan struktur `COORD` dan memanggil Windows API `SetConsoleCursorPosition`. Jika nilai x atau y berada di luar batas jendela terminal, kursor tidak berpindah.

Contoh Penggunaan — C17

```
gotoxy(10, 5);
printf("KASIRKITA POS");
```

2. gotoPrintString()

Signature Bahasa C — lib.h

```
void gotoPrintString(int x, int y, char *text);
```

Deskripsi: Memindahkan kursor ke koordinat `(x, y)` dan langsung mencetak string teks pada posisi tersebut.

© 2026 Hello Project · Hak Cipta Dilindungi

Parameter	Tipe Data	Deskripsi
x	int	Kolom awal penulisan teks.
y	int	Baris penulisan teks.
text	char*	Pointer string yang akan dicetak ke layar (diakhiri NUL).

Contoh Penggunaan C17

```
gotoPrintString(4, 3, "Nama Kasir : Siti Aminah");
gotoPrintString(4, 4, "Waktu Trans: 2026-10-20");
```

3. gotoPrintChar()

Signature Bahasa C lib.h

```
void gotoPrintChar(int x, int y, char ch);
```

Deskripsi: Memindahkan kursor ke posisi (x, y) dan mencetak tepat satu karakter tunggal.

Parameter	Tipe Data	Deskripsi
x, y	int	Koordinat kolom dan baris pencetakan.
ch	char	Karakter tunggal yang ingin ditampilkan.

Contoh Penggunaan C17

```
// Menggambar panah penanda pilihan menu aktif
gotoPrintChar(6, 10, '>');
```

4. borderLine()

Signature Bahasa C lib.h

```
void borderLine(int x, int y, int width, int height);
```

Deskripsi: Menggambar kotak bingkai bergaris tunggal dengan sudut dan tepi ASCII standar.

Parameter	Tipe Data	Deskripsi
x, y	int	Titik koordinat sudut kiri-atas bingkai.
width	int	Lebar bingkai dalam jumlah karakter kolom.
height	int	Tinggi bingkai dalam jumlah karakter baris.

© 2026 Hello Project · Hak Cipta Dilindungi

Contoh Penggunaan C17

```
// Menggambar kotak formulir kasir berukuran 50x12
borderLine(5, 5, 50, 12);
```

5. borderDoubleLine()

Signature Bahasa C lib.h

```
void borderDoubleLine(int x, int y, int width, int height);
```

Deskripsi: Menggambar kotak bingkai bergaris ganda (*double line box*) untuk penekanan visual pada judul utama atau struk nota belanja.

Contoh Penggunaan C17

```
// Bingkai utama menu kasir
borderDoubleLine(2, 1, 76, 22);
```

6. borderClear()

Signature Bahasa C lib.h

```
void borderClear(int x, int y, int width, int height);
```

Deskripsi: Menghapus garis tepi bingkai yang sebelumnya digambar dengan menimpanya menggunakan spasi kosong, tanpa mengganggu isi di bagian dalam kotak.

7. clearArea()

Signature Bahasa C lib.h

```
void clearArea(int x, int y, int width, int height);
```

Deskripsi: Membersihkan seluruh area persegi panjang (mengisi area dengan spasi kosong) dari koordinat `(x, y)` seluas `width × height`.

> **Catatan Perilaku:** Fungsi `clearArea()` pada berkas asli memanggil `showCursor()` di akhir eksekusinya. Jika antarmuka Anda menghendaki kursor tersembunyi (misal di halaman menu), panggil `hideCursor()` kembali setelah menggunakan fungsi ini.

8. hideCursor() & 9. showCursor()

Signature Bahasa C lib.h

```
void hideCursor(void);
void showCursor(void);
```

© 2026 Hello Project · Hak Cipta Dilindungi

Deskripsi: Mengontrol visibilitas kursor kedip terminal Windows menggunakan struktur `CONSOLE_CURSOR_INFO`.

Pola Penggunaan Bersih C17

```
hideCursor(); // Sembunyikan kursor saat user menavigasi menu panah
// ... render menu ...
showCursor(); // Tampilkan kembali kursor saat kasir mengetik di form
```

10. fullscreen(), 11. remove_scrollbar(), & 12. setConsoleFont()

Signature Bahasa C lib.h

```
void fullscreen(void);
void remove_scrollbar(void);
void setConsoleFont(const wchar_t *fontName, int sizeX, int sizeY);
```

Deskripsi: Pengaturan lingkungan jendela console terminal Windows:

- `fullscreen()`: Mengubah jendela terminal menjadi mode layar penuh (*Alt+Enter mode*) via `SetConsoleDisplayMode`.
- `remove_scrollbar()`: Mengunci ukuran buffer layar sama persis dengan ukuran jendela console untuk mematikan *scrollbar* vertikal/horizontal.
- `setConsoleFont()`: Mengubah jenis huruf terminal (misalnya `L"Consolas"`) dan ukuran pikselnya menggunakan `SetCurrentConsoleFontEx`.

13. PrintTxt()

Signature Bahasa C lib.h

```
void PrintTxt(char *filePath, int x, int y);
```

Deskripsi: Membuka berkas teks pada path yang ditentukan, lalu mencetak baris demi baris ke console mulai dari koordinat `(x, y)`.

> **Koreksi Keamanan Wajib:** Pastikan pointer berkas hasil `fopen()` diperiksa tidak bernilai `NULL` sebelum membaca data!

Contoh Penggunaan C17

```
// Menampilkan logo ASCII art toko pada kolom 10, baris 2
PrintTxt("assets/ascii/header_toko.txt", 10, 2);
```

14. scanInputString() & 15. scanInputText()

Signature Bahasa C lib.h

```
void scanInputString(int x, int y, char *text, int maxChar);
void scanInputText(int x, int y, char *text, int maxChar);
```

© 2026 Hello Project · Hak Cipta Dilindungi

Deskripsi: Menerima masukan teks interaktif dari kasir dengan membatasi jumlah pengetikan maksimal sebesar `maxChar`. Menolak tombol fungsi dan tombol panah.

16. scanInputPassword()

```
Signature Bahasa C                                                    lib.h
void scanInputPassword(int x, int y, char *text, int maxChar);
```

Deskripsi: Menangkap input rahasia (seperti PIN otorisasi supervisor atau kasir) dengan menyamarkan setiap karakter yang diketikkan di layar menjadi karakter bintang (*).

17. scanInputNumber()

```
Signature Bahasa C                                                    lib.h
void scanInputNumber(int x, int y, int *num, int maxChar);
```

Deskripsi: Hanya menerima masukan karakter angka (0-9). Karakter alfabetik dan simbol diabaikan secara otomatis sehingga kasir tidak dapat salah menginput huruf pada kolom kuantitas barang. Hasil dikonversi ke variabel bertipe `int`.

```
Contoh Penggunaan                                                     C17
int kuantitas = 0;
gotoPrintString(4, 8, "Jumlah Beli : ");
scanInputNumber(18, 8, &kuantitas, 3); // Maksimal 3 digit (hingga 999)

if (kuantitas <= 0) {
    gotoPrintString(4, 10, "[!] Kuantitas beli harus lebih dari 0!");
}
```

18. scanInputMoney()

```
Signature Bahasa C                                                    lib.h
void scanInputMoney(int x, int y, int *num, int minChar, int maxChar);
```

Deskripsi: Menerima masukan nominal uang pembayaran tunai dengan validasi jumlah digit minimal (`minChar`) dan digit maksimal (`maxChar`).

6.2 Tabel Konstanta Keyboard & Tombol Panah Navigasi

Fungsi penangkap tombol console seperti `_getch()` mengembalikan kode ASCII integer. Memahami nilai konstanta ini sangat penting untuk merancang antarmuka menu yang dapat digerakkan dengan tombol panah:

© 2026 Hello Project · Hak Cipta Dilindungi

Nama Tombol Keyboard	Kode ASCII / Byte Pertama	Kode Byte Kedua (Jika Extended Key)	Fungsi Navigasi pada KasirKita
ENTER	`13`	—	Mengonfirmasi pilihan menu atau input data.
BACKSPACE	`8`	—	Menghapus satu karakter terakhir yang diketik.
TAB	`9`	—	Berpindah ke kolom formulir berikutnya.
ESCAPE (ESC)	`27`	—	Membatalkan operasi atau kembali ke menu sebelumnya.
SPASI	`32`	—	Memberikan spasi karakter.
PANAH ATAS (▲)	`224` atau `0`	`72`	Menggerakkan seleksi menu ke atas.
PANAH BAWAH (▼)	`224` atau `0`	`80`	Menggerakkan seleksi menu ke bawah.
PANAH KIRI (◄)	`224` atau `0`	`75`	Mengurangi kuantitas / halaman sebelumnya.
PANAH KANAN (►)	`224` atau `0`	`77`	Menambah kuantitas / halaman berikutnya.

6.3 Contoh Program Menu Console Lengkap Siap Eksekusi

Berikut adalah implementasi program C lengkap yang menggabungkan bingkai ganda, penulisan teks terformat, penyembunyian kursor, dan navigasi menu interaktif:

© 2026 Hello Project · Hak Cipta Dilindungi

src/menu_demo.c · Program Menu Interaktif KasirKita C17

```c
#include <stdio.h>
#include <stdlib.h>
#include <windows.h>
#include <conio.h>
#include "lib.h"

#define KEY_UP 72
#define KEY_DOWN 80
#define KEY_ENTER 13
#define KEY_ESC 27

void tampilkan_antarmuka(int pilihan_aktif) {
    const char *menu[] = {
        "1. Transaksi Penjualan Kasir",
        "2. Kelola Katalog & Stok Barang",
        "3. Rekap Laporan Keuangan Harian",
        "4. Keluar dari Program"
    };

    // Bersihkan layar & gambar bingkai utama
    system("cls");
    borderDoubleLine(2, 1, 74, 22);

    // Judul Header Toko
    gotoPrintString(26, 3, "=== KASIRKITA POS v1.0 ===");
    gotoPrintString(24, 4, "Toko Alat Tulis & Perlengkapan");

    // Garis pemisah horizontal
    for (int col = 4; col <= 72; ++col) {
        gotoPrintChar(col, 6, '-');
    }

    // Render daftar menu dengan penunjuk aktif
    for (int i = 0; i < 4; ++i) {
        if (i == pilihan_aktif) {
            gotoPrintString(10, 9 + (i * 2), "> [X] ");
            gotoPrintString(16, 9 + (i * 2), (char*)menu[i]);
        } else {
            gotoPrintString(10, 9 + (i * 2), "  [ ] ");
            gotoPrintString(16, 9 + (i * 2), (char*)menu[i]);
        }
    }

    // Petunjuk Navigasi di Bagian Bawah
    gotoPrintString(6, 19, "Gunakan tombol panah [▲ / ▼] dan [ENTER] untuk memilih.");
    gotoPrintString(6, 20, "Tekan [ESC] untuk keluar.");
}

int main(void) {
    int pilihan = 0;
    int tombol = 0;

    remove_scrollbar();
    hideCursor();

    while (1) {
        tampilkan_antarmuka(pilihan);
```

```
        tombol = _getch();

        if (tombol == 224 || tombol == 0) { // Tombol panah (extended key)
            tombol = _getch();
            if (tombol == KEY_UP && pilihan > 0) {
                pilihan--;
            } else if (tombol == KEY_DOWN && pilihan < 3) {
                pilihan++;
            }
        } else if (tombol == KEY_ENTER) {
            if (pilihan == 3) break; // Keluar

            // Eksekusi menu yang dipilih
            clearArea(5, 8, 68, 10);
            gotoPrintString(10, 10, "Anda memilih menu!");
            gotoPrintString(10, 12, "Tekan tombol apa saja untuk kembali...");
            _getch();
        } else if (tombol == KEY_ESC) {
            break;
        }
    }

    showCursor();
    system("cls");
    printf("Terima kasih telah menggunakan KasirKita.\n");
    return 0;
}
```

© 2026 Hello Project · Hak Cipta Dilindungi

MODUL 04: UI/UX & ASCII ART

BAB 7: Desain UI/UX Terminal Console & ASCII Art

7.1 Prinsip Desain UI/UX Antarmuka Teks (TUI)

Antarmuka berbasis teks (*Text-based User Interface - TUI*) pada jendela console terminal memiliki keterbatasan ruang piksel fisik jika dibandingkan antarmuka grafis (GUI) modern. Namun, keterbatasan tersebut justru melahirkan seni desain visual yang khas, bersih, dan berkarakter jika dirancang dengan prinsip ergonomi perangkat lunak yang matang.

Hierarki Visual Berbasis Garis

Gunakan bingkai ganda (`borderDoubleLine`) untuk komponen penampung paling luar dan judul toko, serta bingkai tunggal (`borderLine`) untuk formulir input atau tabel rincian transaksi.

Konsistensi Ruang & Margin Layar

Sisakan batas tepi (*padding*) minimal 2 kolom dari tepi kiri dan kanan jendela terminal 80 kolom agar antarmuka tidak terasa sesak atau menabrak bezel monitor kasir.

7.2 Persiapan & Sketsa Gambar untuk ASCII Art

Sebelum mengubah citra menjadi seni karakter teks, penting untuk memahami karakteristik gambar yang cocok untuk diubah menjadi ASCII art pada resolusi terminal kasir:

- **Pilih Gambar Bersiluet Tegas:** Gambar dengan kontras tinggi hitam-putih murni (seperti logo pena, buku, atau gunting) akan menghasilkan karakter teks yang jauh lebih jelas terbaca daripada foto realistik berpiksel halus.
- **Rasio Aspek Font Monospace:** Pada terminal komputer, tinggi karakter font rata-rata dua kali lipat lebih besar daripada lebarnya (aspek rasio sekitar 1:2). Citra sumber yang berbentuk lingkaran sempurna akan tampak lonjong vertikal jika tidak dikompensasi saat konversi.

7.3 Panduan Konversi Citra ke Karakter via ASCIIArt.eu

Untuk menghasilkan banner nama toko atau logo kasir yang indah, kita menggunakan layanan utilitas web **ASCIIArt.eu** (atau generator serupa):

© 2026 Hello Project · Hak Cipta Dilindungi

Langkah Konversi	Tindakan & Pengaturan Parameter Rekomendasi
1. Pemilihan Citra	Siapkan file logo sederhana berformat PNG/JPG dengan latar belakang putih atau transparan.
2. Unggah Gambar	Buka menu *Image to ASCII Converter* pada situs ASCIIArt.eu dan unggah gambar logo toko ATK.
3. Pengaturan Lebar (Width)	**Kunci Pengaturan Terpenting:** Atur parameter *Character Width* pada angka **50 hingga 65 karakter**. Jangan pernah memilih lebar di atas 74 karakter! Jendela console standar memiliki batas 80 karakter. Jika lebar teks melebihi batas, baris akan membungkus (*wrapping*) dan menghancurkan tampilan gambar!
4. Mode Karakter & Kontras	Pilih set karakter standar (ASCII Standard). Geser slider kontras hingga bagian latar belakang benar-benar bersih dan hanya logo yang terbentuk dari karakter padat seperti #, @, dan %.
5. Penyimpanan Aset	Salin teks hasil konversi dan simpan ke dalam berkas teks di folder `assets/ascii/header_toko.txt`.

7.4 Teknik Integrasi ASCII Art pada Program C (Escaping Rules)

Terdapat dua metode utama untuk menampilkan seni ASCII art ke dalam program C:

Metode 1: Pembacaan Berkas Eksternal via PrintTxt() (Sangat Direkomendasikan)

Menyimpan seni ASCII pada berkas teks eksternal (`.txt`) lalu membacanya menggunakan fungsi `PrintTxt()` dari `lib.h` adalah pendekatan terbersih. Anda dapat mengedit tampilan banner toko menggunakan aplikasi teks editor tanpa perlu melakukan kompilasi ulang (*recompile*) kode C!

assets/ascii/header_toko.txt · Berkas Seni ASCII Eksternal — Plain Text

```
  _  __          _       _  ___ _
 | |/ /         (_)     | |/ (_) |
 | ' / __ _ ___  _ _ __ | ' / _| |_ __ _
 |  < / _` / __|| | '__||  < | | __/ _` |
 | . \ (_| \__ \| | |   | . \| | || (_| |
 |_|\_\__,_|___/|_|_|   |_|\_\_|\__\__,_|
  === PERLENGKAPAN ALAT TULIS KANTOR ===
```

src/render.c · Pemanggilan PrintTxt — C17

```c
// Menampilkan logo toko mulai dari kolom 18, baris 2
PrintTxt("assets/ascii/header_toko.txt", 18, 2);
```

Metode 2: Penyematan String Literal Langsung (Hardcoded Array)

Jika Anda ingin menanamkan teks ASCII langsung di dalam berkas `main.c`, Anda wajib mematuhi **Aturan Escaping Karakter Khusus Bahasa C**:

© 2026 Hello Project · Hak Cipta Dilindungi

Aturan Wajib Escaping Karakter C:

- **Karakter Backslash (\):** Karakter backslash adalah penanda *escape sequence* di C. Setiap kemunculan karakter \ pada seni ASCII wajib digandakan menjadi \\!
- **Karakter Tanda Petik Ganda ("):** Wajib ditulis dengan awalan backslash menjadi \" agar kompiler tidak menganggapnya sebagai penutup string literal.
- **Karakter Tanda Persen (%):** Jika string dicetak menggunakan `printf()`, tanda persen wajib digandakan menjadi %% agar tidak diartikan sebagai penentu format data (*format specifier*).

Contoh String Literal ASCII dengan Escaping Benar C17

```
const char *logo[] = {
    "  _  __          _       _  ___ _        ",
    " | |/ /         (_)     | |/ (_) |       ",
    " | ' / __ _ ___  _ _ __ | ' / _| |_ __ _ ",
    " |  < / _` / __|| | '__||  < | | __/ _` |",
    " | . \\ (_| \__ \| | |   | . \| | || (_| |", // Perhatikan tanda \\
    " |_|\_\__,_|___/|_|_|   |_|\_\_|\__\__,_|"
};

for (int row = 0; row < 6; ++row) {
    gotoPrintString(18, 2 + row, (char*)logo[row]);
}
```

7.5 Desain Tata Letak Layar Kasir: Form & Struk Nota Belanja

Berikut adalah rancangan tata letak visual struk nota belanja terminal yang dihasilkan KasirKita saat transaksi belanja berhasil disahkan:

Tata Letak Struk Nota Belanja Terminal (ASCII Receipt) TUI Output

```
      ====================================================
                        KASIRKITA
                   TOKO PERLENGKAPAN ATK
              Jl. Pendidikan No. 45, Jakarta
      ====================================================
      No. Nota : NOT-20261020-001      Kasir : Siti
      Tanggal  : 2026-10-20 14:35      Cust  : Rani
      ----------------------------------------------------
      ITEM                   QTY    HARGA SATUAN  SUBTOTAL
      ----------------------------------------------------
      Buku Tulis A5            2    Rp 12.000     Rp 24.000
      ----------------------------------------------------
      TOTAL TAGIHAN        :               Rp 24.000
      TUNAI DITERIMA       :               Rp 30.000
      UANG KEMBALIAN       :               Rp  6.000
      ====================================================
             Terima Kasih Atas Kunjungan Anda!
           Barang yang dibeli tidak dapat ditukar
      ====================================================
```

7.6 Simulator Terminal Studio 80×25 & Uji Keterbacaan Antarmuka

Panduan Uji Antarmuka Console Kasir:

- ✓ Pastikan tidak ada teks formulir yang melewati kolom ke-78 agar tidak menabrak batas kanan console.
- ✓ Periksa kontras warna: gunakan kombinasi warna teks terang (putih/kuning/hijau) di atas latar belakang gelap terminal (hitam/navy) agar kasir tidak lelah membaca layar berjam-jam.
- ✓ Pastikan kursor hanya muncul pada kolom pengetikan data yang sedang aktif, dan tersembunyi saat berada di menu utama.

© 2026 Hello Project · Hak Cipta Dilindungi

MODUL KOLABORASI: GIT & GITHUB

BAB 8: Manajemen Kolaborasi Tim dengan Git & GitHub

8.1 Urgensi Sistem Kontrol Versi (VCS) dalam Kerja Kelompok

Penyebab kegagalan nomor satu dalam proyek tugas besar pemrograman kelompok bukanlah kerumitan logika sintaks bahasa C, melainkan **ketiadaan koordinasi kode dan manajemen versi yang buruk**.

Tanpa sistem kontrol versi (*Version Control System - VCS*), mahasiswa sering saling mengirimkan file arsip zip melalui aplikasi pesan singkat WhatsApp atau flashdisk dengan penamaan berantakan seperti `kasirkita_final.zip`, `kasirkita_final_beneran.zip`, hingga `kasirkita_final_fix_beneran_tuntas_v2.zip`. Praktik ini hampir selalu berujung pada hilangnya kode teman sekelompok, fitur yang tertimpa secara tidak sengaja, dan kepanikan massal menjelang batas waktu penyerahan (*deadline*).

Pencegahan Tabrakan Kode

Git memungkinkan 3 hingga 5 anggota tim bekerja secara mandiri pada berkas atau fungsi C yang berbeda secara bersamaan, lalu menggabungkannya ke dalam cabang utama secara rapi dan sistematis.

Transparansi & Bukti Kontribusi

Di GitHub, setiap baris kode yang diubah memiliki stempel waktu dan identitas penulis yang jelas (*commit history*). Ini menjadi bukti otentik penilaian dosen mengenai kontribusi nyata masing-masing mahasiswa.

8.2 Memahami 4 Wilayah Kerja Git

File di komputer tidak langsung loncat ke server GitHub di awan saat Anda menekan tombol simpan (*Ctrl+S*). Git membagi siklus hidup berkas ke dalam **4 wilayah kerja terpisah**:

Wilayah Kerja	Lokasi Fisik	Fungsi & Analogi Sehari-Hari	Perintah Terkait
1. Working Directory	Folder proyek lokal di laptop Anda.	**Meja Kerja:** Tempat Anda mengetik dan mengedit kode C di IntelliJ IDEA. File yang berubah berstatus *Modified* (belum dilacak).	`git status`
2. Staging Area (Index)	Ruang memori sementara Git.	**Kotak Paket Pengiriman:** Tempat Anda memilah berkas mana saja yang sudah matang dan siap dibungkus menjadi satu catatan komit.	`git add <file>`
3. Local Repository	Database internal Git di folder `.git/`.	**Lemari Arsip Pribadi:** Catatan permanen di laptop Anda. Memiliki nomor hash komit unik dan pesan ringkasan pekerjaan.	`git commit -m "..."`
4. Remote Repository	Server cloud di GitHub.com.	**Arsip Bersama Tim di Awan:** Wadah sinkronisasi tempat seluruh anggota kelompok saling bertukar dan menggabungkan pekerjaan.	`git push` / `git pull`

© 2026 Hello Project · Hak Cipta Dilindungi

8.3 Setup Identitas & Inisialisasi Repository Bersama

Sebelum mulai berkolaborasi, setiap mahasiswa wajib menyetel identitas nama dan email pada terminal Git komputer masing-masing:

Terminal PowerShell · Konfigurasi Identitas Git Global — Bash / PowerShell

```
# 1. Menyetel identitas nama lengkap dan email GitHub resmi
git config --global user.name "Budi Santoso"
git config --global user.email "budi.santoso@students.ac.id"

# 2. Mengkloning repositori proyek kelompok dari GitHub ke laptop
git clone https://github.com/kelompok-kasirkita/kasirkita-pos.git

# 3. Masuk ke direktori proyek
cd kasirkita-pos
```

8.4 Strategi Feature Branching: Bekerja Paralel Tanpa Tabrakan

Aturan Emas Tim Profesional: Cabang utama (`main`) adalah cabang suci! Dilarang keras melakukan pengetikan kode dan komit langsung pada cabang `main`. Cabang `main` hanya boleh memuat kode yang sudah stabil, bebas dari error kompilasi, dan siap didemonstrasikan.

> **Alur Kerja Feature Branching:**
> Setiap anggota tim yang mendapat tugas baru wajib membuat cabang fitur (*Feature Branch*) sendiri yang terisolasi dari rekan lainnya:

Terminal · Membuat & Berpindah ke Cabang Fitur — Git CLI

```
# 1. Pastikan Anda berada di cabang main yang paling mutakhir
git checkout main
git pull origin main

# 2. Buat cabang baru khusus untuk fitur Anda (contoh: form input produk)
git checkout -b feat/form-produk

# 3. Sekarang Anda bebas mengedit kode tanpa takut merusak pekerjaan rekan lain!
```

Format Nama Cabang	Tujuan Pembuatan	Contoh Nama Cabang
`feat/<nama-fitur>`	Pengembangan fitur atau modul baru	`feat/keranjang-belanja`, `feat/cetak-nota`
`fix/<nama-bug>`	Perbaikan cacat logika atau bug sistem	`fix/kembalian-negatif`, `fix/buffer-overflow-nama`
`docs/<nama-dokumen>`	Pembaruan dokumen AP, Probis, atau UAT	`docs/laporan-akhir`, `docs/revisi-kamus-data`

8.5 Rutinitas Harian: Aturan Emas Push, Pull, dan Commit

Agar proyek kelompok berjalan mulus tanpa konflik, patuhi siklus kerja harian 5 langkah berikut:

© 2026 Hello Project · Hak Cipta Dilindungi

Siklus 5 Langkah Kerja Harian Git — Git CLI

```
# LANGKAH 1: Saat baru buka laptop / sebelum mulai koding, tarik kode terbaru rekan
git pull --rebase origin main

# LANGKAH 2: Periksa berkas apa saja yang baru Anda modifikasi
git status

# LANGKAH 3: Daftarkan berkas yang siap dikomit ke Staging Area
git add src/transaksi.c include/lib.h

# LANGKAH 4: Buat komit dengan pesan deskriptif yang bermakna
git commit -m "feat(transaksi): tambah validasi batas uang tunai kurang"

# LANGKAH 5: Unggah cabang fitur Anda ke server GitHub
git push -u origin feat/form-produk
```

Standar Pesan Komit (Conventional Commits):
Hindari pesan komit pemalas seperti `"update"`, `"revisi"`, atau `"fix bismillah"`. Gunakan format baku: `<tipe>(<ruang-lingkup>): <penjelasan singkat>`.
Contoh: `feat(ui): gambar banner logo toko dengan ASCII art` atau `fix(input): perbaiki bug backspace pada scanInputNumber`.

8.6 Anatomi & Resolusi Merge Conflict dengan Kepala Dingin

Merge Conflict (Konflik Penggabungan) bukanlah kesalahan fatal perangkat lunak! Konflik adalah mekanisme proteksi cerdas dari Git yang memberi tahu Anda: "*Dua anggota kelompok mengubah baris kode yang sama pada saat yang sama, dan Git tidak ingin menebak kode siapa yang harus dipakai.*"

Mengenali Anatomi Simbol Penanda Konflik

Anatomi Berkas yang Mengalami Merge Conflict — Source Diff

```
<<<<<<< HEAD (Perubahan pada Cabang Milik Anda)
    // Versi Anda: Menghitung kembalian langsung
    kembalian = bayar - total;
    printf("Kembalian Anda: Rp %d\n", kembalian);
=======
    // Versi Rekan Tim di GitHub: Menyertakan validasi uang kurang
    if (bayar < total) {
        printf("Uang tunai tidak mencukupi!\n");
        return;
    }
    kembalian = bayar - total;
>>>>>>> main (Perubahan dari Rekan Kelompok)
```

Langkah Resolusi Konflik	Tindakan yang Wajib Dilakukan
1. Jangan Panik	Jangan menghapus folder proyek atau melakukan `git clone` ulang! Konflik adalah kejadian normal dalam kerja tim.
2. Komunikasi Tim	Hubungi rekan tim yang menulis kode pembanding. Diskusikan logika mana yang paling benar atau bagaimana menggabungkan kedua versi tersebut.
3. Bersihkan Penanda	Buka file di IntelliJ IDEA, hapus baris penanda `<<<<<<<`, `=======`, dan `>>>>>>>`, lalu pertahankan kode gabungan yang benar.
4. Kompilasi & Uji	Lakukan kompilasi ulang (`gcc`) untuk membuktikan bahwa kode hasil penggabungan bebas dari galat sintaks.
5. Komit Resolusi	Jalankan `git add .` lalu `git commit -m "merge: selesaikan konflik validasi pembayaran"` dan lakukan `git push`.

8.7 Standar Pull Request (PR) & Praktik Terbaik Code Review

Setelah fitur pada cabang `feat/form-produk` selesai diuji secara lokal, proses penggabungan ke cabang `main` dilakukan melalui **Pull Request (PR)** di web GitHub:

1. Buka repositori tim di GitHub, lalu klik tombol *Compare & Pull Request*.
2. Tuliskan ringkasan fitur yang dibuat dan lampirkan bukti tangkapan layar pengujian (screenshot terminal).
3. Minta salah seorang rekan kelompok untuk bertindak sebagai *Reviewer*. Rekan tim akan memeriksa baris demi baris kode untuk memastikan tidak ada logika yang melanggar spesifikasi.
4. Setelah disetujui (*Approved*), klik tombol **Merge Pull Request**. Kode kini resmi tergabung ke cabang utama `main`!

8.8 Template .gitignore Standar C di IntelliJ IDEA

Salah satu kesalahan paling memalukan yang sering dilakukan mahasiswa adalah mengunggah file binary executable (`.exe`), folder cache IDE (`.idea/`), atau file sementara ke repositori GitHub. Hal ini membuat ukuran repositori membengkak dan memicu konflik terus-menerus.

Simpan file berikut dengan nama persis **`.gitignore`** di folder utama (*root*) proyek:

© 2026 Hello Project · Hak Cipta Dilindungi

.gitignore · Template Standar Proyek C KasirKita di IntelliJ Git Configuration

```
# ============================================================================
# Konfigurasi Internal IDE (IntelliJ IDEA / CLion / VSCode)
# ============================================================================
.idea/
cmake-build-*/
*.iws
*.iml
.vscode/

# ============================================================================
# Berkas Binary Executable & Hasil Kompilasi
# ============================================================================
build/
bin/
out/
*.exe
*.o
*.obj
*.dll
*.so

# ============================================================================
# Berkas Basis Data Sementara & Log
# ============================================================================
*.sqlite
*.sqlite3
*.log
*.tmp
*.bak

# ============================================================================
# Catatan Sistem Operasi
# ============================================================================
Thumbs.db
Desktop.ini
.DS_Store
```

8.9 Git Emergency Cheat Sheet & Penanganan Kondisi Darurat

© 2026 Hello Project · Hak Cipta Dilindungi

Situasi Darurat	Perintah Penyelamat Git	Efek & Keterangan
Salah komit (belum di-push)	`git reset --soft HEAD~1`	Membatalkan komit terakhir, namun semua ketikan kode Anda tetap aman di Staging Area untuk diperbaiki.
Harus pindah branch tapi kodingan belum selesai	`git stash` *kembali lagi:* `git stash pop`	Menyimpan pekerjaan sementara ke dalam kantong tersembunyi tanpa harus dikomit. Anda bisa pindah branch dengan aman, lalu mengambil kembali pekerjaan Anda kapan saja.
Salah edit dan ingin mengembalikan file ke kondisi asal	`git restore src/main.c`	Membuang perubahan lokal yang belum di-stage dan mengembalikan file persis seperti komit terakhir.
Melihat riwayat komit dengan grafik cabang	`git log --oneline --graph --decorate`	Menampilkan pohon riwayat perubahan tim secara visual dan ringkas dalam satu baris per komit.

© 2026 Hello Project · Hak Cipta Dilindungi

LAMPIRAN & KODE SUMBER

BAB 9: Lampiran & Kode Sumber Referensi

9.1 Kode Lengkap Header lib.h Terkoreksi (Clean Patched Version)

Berikut adalah kode sumber lengkap pustaka `include/lib.h` yang telah diperbaiki dari seluruh cacat logika (bug backspace, alokasi karakter, proteksi berkas, dan pemeriksaan pointer) sehingga siap dikompilasi secara bersih pada kompiler GCC C17 modern:

© 2026 Hello Project · Hak Cipta Dilindungi

include/lib.h · Versi Bersih Terkoreksi (Production-Ready) Bahasa C17

```
#ifndef LIB_H
#define LIB_H

#include <stdio.h>
#include <stdlib.h>
#include <windows.h>
#include <conio.h>
#include <wchar.h>
#include <ctype.h>

// Konstanta Tombol Keyboard
#define KEY_ENTER 13
#define KEY_BACKSPACE 8
#define KEY_TAB 9
#define KEY_ESC 27
#define KEY_SPACE 32

// 1. gotoxy: Memindahkan posisi kursor
void gotoxy(int x, int y) {
    COORD coord;
    coord.X = (SHORT)x;
    coord.Y = (SHORT)y;
    SetConsoleCursorPosition(GetStdHandle(STD_OUTPUT_HANDLE), coord);
}

// 2. gotoPrintString: Cetak string pada koordinat tertentu
void gotoPrintString(int x, int y, char *text) {
    gotoxy(x, y);
    printf("%s", text);
}

// 3. gotoPrintChar: Cetak satu karakter pada koordinat tertentu
void gotoPrintChar(int x, int y, char ch) {
    gotoxy(x, y);
    putchar(ch);
}

// 4. borderLine: Gambar bingkai garis tunggal
void borderLine(int x, int y, int width, int height) {
    for (int i = x; i <= x + width; ++i) {
        gotoPrintChar(i, y, '-');
        gotoPrintChar(i, y + height, '-');
    }
    for (int j = y; j <= y + height; ++j) {
        gotoPrintChar(x, j, '|');
        gotoPrintChar(x + width, j, '|');
    }
    gotoPrintChar(x, y, '+');
    gotoPrintChar(x + width, y, '+');
    gotoPrintChar(x, y + height, '+');
    gotoPrintChar(x + width, y + height, '+');
}

// 5. borderDoubleLine: Gambar bingkai garis ganda
void borderDoubleLine(int x, int y, int width, int height) {
    for (int i = x; i <= x + width; ++i) {
        gotoPrintChar(i, y, '=');
```

```
        gotoPrintChar(i, y + height, '=');
    }
    for (int j = y; j <= y + height; ++j) {
        gotoPrintChar(x, j, '#');
        gotoPrintChar(x + width, j, '#');
    }
    gotoPrintChar(x, y, '#');
    gotoPrintChar(x + width, y, '#');
    gotoPrintChar(x, y + height, '#');
    gotoPrintChar(x + width, y + height, '#');
}

// 6. borderClear: Menghapus tepi bingkai
void borderClear(int x, int y, int width, int height) {
    for (int i = x; i <= x + width; ++i) {
        gotoPrintChar(i, y, ' ');
        gotoPrintChar(i, y + height, ' ');
    }
    for (int j = y; j <= y + height; ++j) {
        gotoPrintChar(x, j, ' ');
        gotoPrintChar(x + width, j, ' ');
    }
}

// 7. clearArea: Mengosongkan area persegi panjang
void clearArea(int x, int y, int width, int height) {
    for (int j = y; j <= y + height; ++j) {
        gotoxy(x, j);
        for (int i = 0; i <= width; ++i) {
            putchar(' ');
        }
    }
}

// 8. hideCursor: Menyembunyikan kursor
void hideCursor(void) {
    HANDLE consoleHandle = GetStdHandle(STD_OUTPUT_HANDLE);
    CONSOLE_CURSOR_INFO info;
    info.dwSize = 100;
    info.bVisible = FALSE;
    SetConsoleCursorInfo(consoleHandle, &info);
}

// 9. showCursor: Menampilkan kursor
void showCursor(void) {
    HANDLE consoleHandle = GetStdHandle(STD_OUTPUT_HANDLE);
    CONSOLE_CURSOR_INFO info;
    info.dwSize = 20;
    info.bVisible = TRUE;
    SetConsoleCursorInfo(consoleHandle, &info);
}

// 10. fullscreen: Mode layar penuh
void fullscreen(void) {
    SetConsoleDisplayMode(GetStdHandle(STD_OUTPUT_HANDLE), CONSOLE_FULLSCREEN_MODE, 0);
}

// 11. remove_scrollbar: Hilangkan bilah gulung terminal
void remove_scrollbar(void) {
```

© 2026 Hello Project · Hak Cipta Dilindungi

```
    HANDLE handle = GetStdHandle(STD_OUTPUT_HANDLE);
    CONSOLE_SCREEN_BUFFER_INFO info;
    GetConsoleScreenBufferInfo(handle, &info);
    COORD new_size = {
        info.srWindow.Right - info.srWindow.Left + 1,
        info.srWindow.Bottom - info.srWindow.Top + 1
    };
    SetConsoleScreenBufferSize(handle, new_size);
}

// 12. setConsoleFont: Ubah jenis font console
void setConsoleFont(const wchar_t *fontName, int sizeX, int sizeY) {
    HANDLE hOut = GetStdHandle(STD_OUTPUT_HANDLE);
    CONSOLE_FONT_INFOEX cfi;
    cfi.cbSize = sizeof(cfi);
    cfi.nFont = 0;
    cfi.dwFontSize.X = sizeX;
    cfi.dwFontSize.Y = sizeY;
    cfi.FontFamily = FF_DONTCARE;
    cfi.FontWeight = FW_NORMAL;
    wcsncpy(cfi.FaceName, fontName, LF_FACESIZE - 1);
    cfi.FaceName[LF_FACESIZE - 1] = L'\0';
    SetCurrentConsoleFontEx(hOut, FALSE, &cfi);
}

// 13. PrintTxt: Membaca dan menampilkan berkas teks (Terkoreksi Aman)
void PrintTxt(char *filePath, int x, int y) {
    FILE *fp = fopen(filePath, "r");
    if (fp == NULL) return; // Proteksi keamanan pointer

    char buffer[256];
    int currentY = y;
    while (fgets(buffer, sizeof(buffer), fp) != NULL) {
        gotoPrintString(x, currentY++, buffer);
    }
    fclose(fp);
}

// 14. scanInputString: Input string dengan proteksi backspace terkoreksi
void scanInputString(int x, int y, char *text, int maxChar) {
    showCursor();
    gotoxy(x, y);
    int i = 0;
    char ch;
    while (1) {
        ch = (char)_getch();
        if (ch == KEY_ENTER && i > 0) {
            text[i] = '\0';
            break;
        } else if (ch == KEY_BACKSPACE && i > 0) {
            --i;
            text[i] = '\0'; // Perbaikan tipe C
            printf("\b \b");
        } else if (ch == KEY_ESC) {
            text[0] = '\0';
            break;
        } else if (isprint(ch) && i < maxChar) {
            text[i++] = ch;
            putchar(ch);
```

© 2026 Hello Project · Hak Cipta Dilindungi

```
        }
    }
    text[i] = '\0';
}

// 15. scanInputNumber: Khusus angka integer
void scanInputNumber(int x, int y, int *num, int maxChar) {
    char buffer[64] = {0};
    showCursor();
    gotoxy(x, y);
    int i = 0;
    char ch;
    while (1) {
        ch = (char)_getch();
        if (ch == KEY_ENTER && i > 0) {
            buffer[i] = '\0';
            *num = atoi(buffer);
            break;
        } else if (ch == KEY_BACKSPACE && i > 0) {
            --i;
            buffer[i] = '\0';
            printf("\b \b");
        } else if (isdigit(ch) && i < maxChar) {
            buffer[i++] = ch;
            putchar(ch);
        }
    }
}

#endif // LIB_H
```

9.2 Skrip Build Otomatis PowerShell & Batch

build.ps1 · Skrip Kompilasi Otomatis Windows PowerShell — PowerShell

```
# Skrip Kompilasi Satu Klik KasirKita
Write-Host "Mengompilasi KasirKita POS C17..." -ForegroundColor Cyan

if (-not (Test-Path "build")) {
    New-Item -ItemType Directory -Path "build" | Out-Null
}

gcc -std=c17 -Wall -Wextra src/main.c -Iinclude -o build/kasirkita.exe

if ($LASTEXITCODE -eq 0) {
    Write-Host "[OK] Kompilasi Sukses! Menjalankan program..." -ForegroundColor Green
    & .\build\kasirkita.exe
} else {
    Write-Host "[ERROR] Kompilasi Gagal. Periksa pesan error di atas." -ForegroundColor Red
}
```

9.3 Glosarium Istilah Rekayasa Perangkat Lunak & Pemrograman C

© 2026 Hello Project · Hak Cipta Dilindungi

Istilah Teknis	Definisi & Konteks Penggunaan
Atomisitas (Atomicity)	Prinsip di mana serangkaian operasi diperlakukan sebagai satu kesatuan utuh: semua berhasil (commit) atau semua dibatalkan (rollback).
Rollback	Operasi pengembalian status basis data atau berkas ke kondisi sebelum transaksi dimulai saat terdeteksi galat sistem.
Data Master	Entitas data acuan yang bersifat statis dan digunakan berulang kali (contoh: katalog produk dan kategori).
Data Transaksi	Catatan kejadian bisnis pada titik waktu tertentu yang bersifat dinamis dan bertambah terus (contoh: nota penjualan).
Normalisasi (1NF, 2NF, 3NF)	Teknik perancangan basis data untuk meminimalkan redundansi data dan mencegah anomali pembaruan, penyisipan, dan penghapusan.
Denormalisasi Terkontrol	Penyimpangan sengaja dari kaidah normalisasi murni untuk mengoptimalkan performa query (contoh: menyimpan total belanja dan snapshot harga).
DFD (Data Flow Diagram)	Diagram grafis yang memetakan perpindahan informasi dari entitas eksternal, proses komputasi, hingga media penyimpanan berkas.
UAT (User Acceptance Testing)	Pengujian fungsionalitas dan ergonomi perangkat lunak yang dilakukan langsung oleh calon pengguna akhir sistem di lapangan.
Defect / Bug	Ketidaksesuaian antara hasil aktual yang dihasilkan program dengan hasil yang diharapkan sesuai spesifikasi kebutuhan.
TUI (Text User Interface)	Antarmuka pengguna berbasis karakter teks terminal yang memanfaatkan posisi koordinat kursor dan karakter bingkai ASCII.
Monospace Font	Jenis huruf di mana setiap karakter alfabetik, numerik, maupun simbol memiliki lebar horizontal yang persis sama.
Staging Area	Ruang transit memori Git untuk memilih dan mempersiapkan berkas mana saja yang akan dibungkus ke dalam komit berikutnya.
Merge Conflict	Kondisi di mana Git mendeteksi dua perubahan yang bertentangan pada baris kode yang sama saat menggabungkan dua cabang cabang berbeda.
Conventional Commits	Standar penulisan pesan komit yang terstruktur untuk memudahkan pelacakan riwayat pekerjaan tim kelompok.

© 2026 Hello Project · Hak Cipta Dilindungi